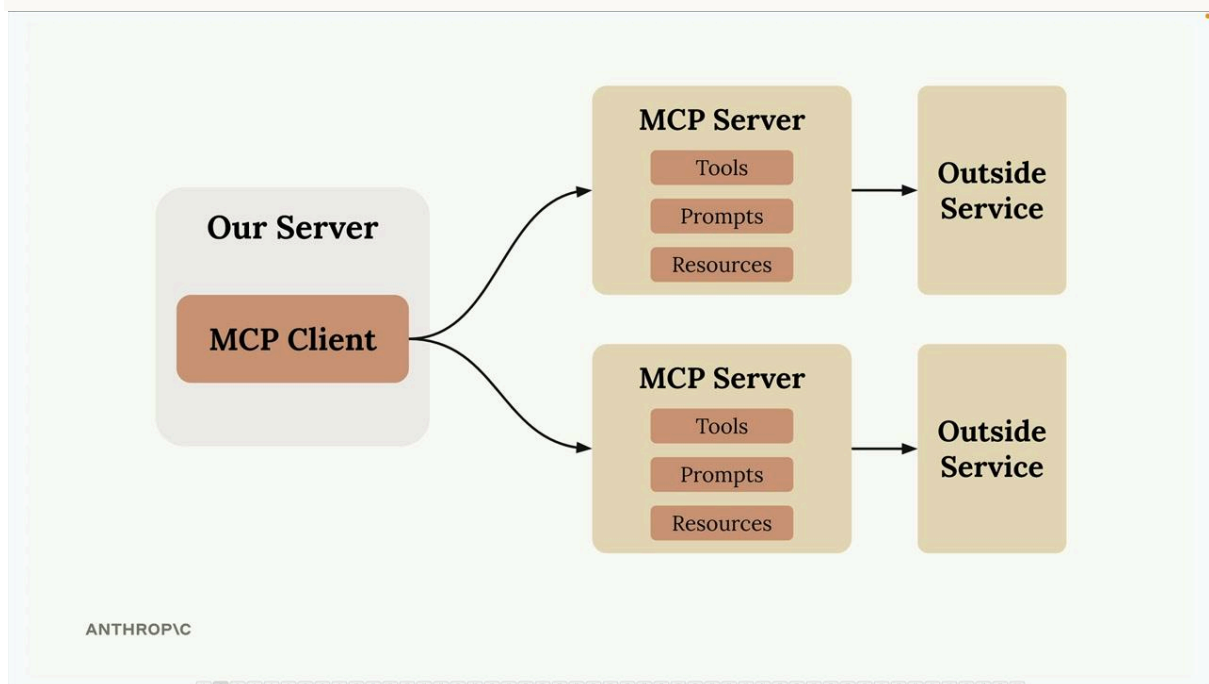


Introducing MCP

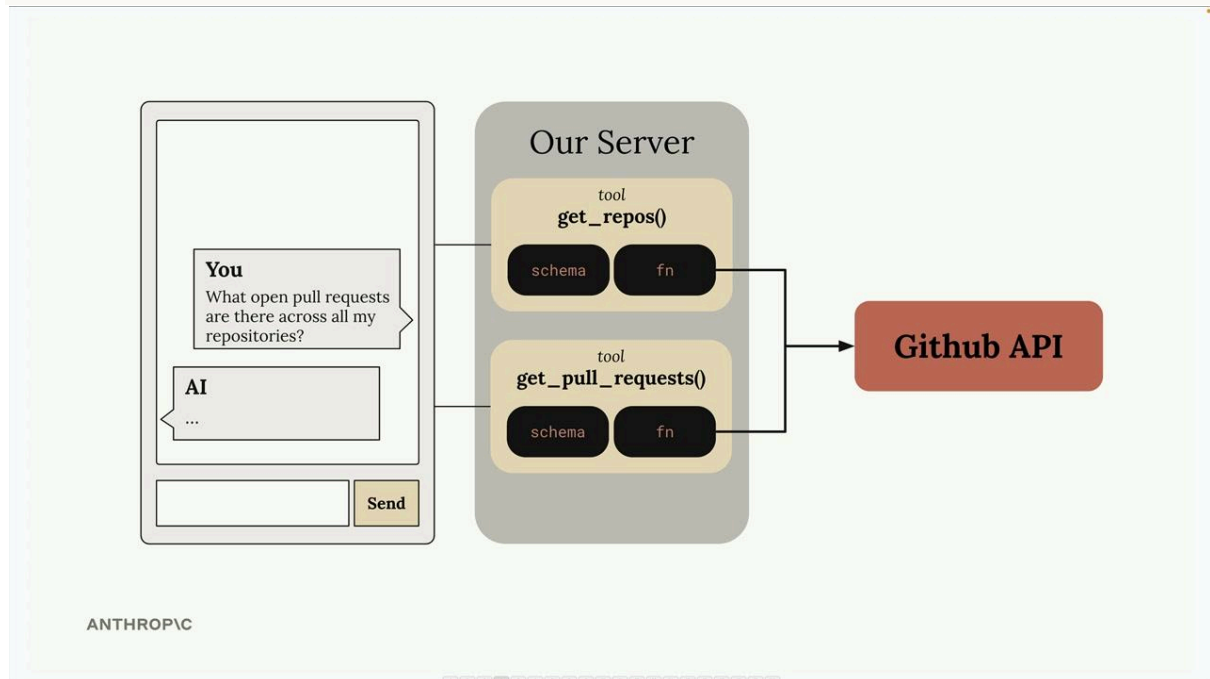
Model Context Protocol (MCP) is a communication layer that provides Claude with context and tools without requiring you to write a bunch of tedious integration code. Think of it as a way to shift the burden of tool definitions and execution away from your server to specialized MCP servers.



When you first encounter MCP, you'll see diagrams showing the basic architecture: an MCP Client (your server) connecting to MCP Servers that contain tools, prompts, and resources. Each MCP server acts as an interface to some outside service.

Understanding MCP Through a Real Example

Let's say you're building a chat interface where users can ask Claude about their GitHub data. A user might ask "What open pull requests are there across all my repositories?" To answer this, Claude needs tools to access GitHub's API.



Without MCP, you'd need to create all the GitHub integration tools yourself. This means writing schemas and functions for every piece of GitHub functionality you want to support.

The Tool Function Problem

GitHub has massive functionality - repositories, pull requests, issues, projects, and much more. To build a complete GitHub chatbot, you'd need to author an incredible number of tools:

Tool Functions

- To handle all of Github's functionality, we'd have to create an incredible number of tool schemas and functions
- **This is all code that we (developers) have to write, test, and maintain**

ANTHROPIC



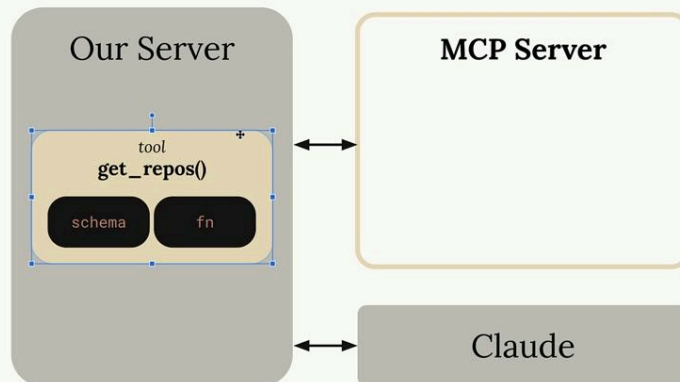
Each tool requires both a schema definition and a function implementation. This represents a lot of code that you have to write, test, and maintain as a developer.

How MCP Solves This

MCP shifts the burden of tool definitions and execution from your server to MCP servers. Instead of you writing all those GitHub tools, they're authored and executed inside a dedicated MCP server.

Model Context Protocol

Shifts the burden of tool definitions and execution onto **MCP Servers**.

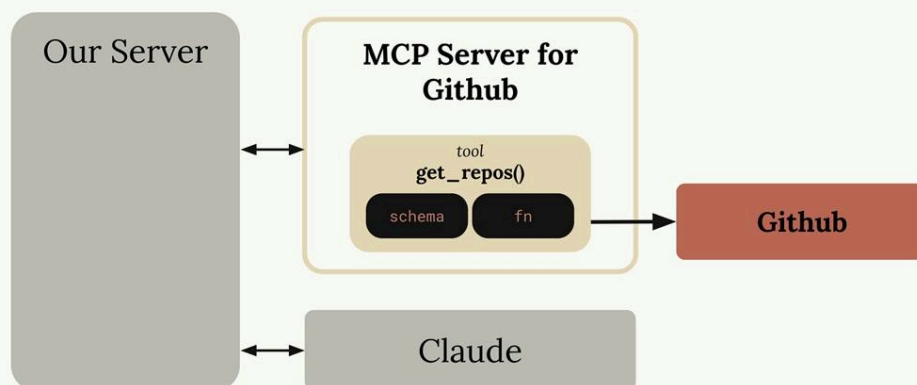


ANTHROPIC

The MCP server acts as a wrapper around GitHub's functionality, providing pre-built tools that you can use without having to implement them yourself.

MCP Servers

MCP Servers provide access to data or functionality implemented by some outside service.



ANTHROPIC

MCP servers provide access to data or functionality implemented by outside services. They package up complex integrations into reusable components that any application can connect to.

Common Questions About MCP

Common Questions	
Who authors the MCP Server?	Anyone! Often the service provider itself will make their own MCP implementation. You can make a MCP server to wrap up access to some service.
How is using an MCP Server different from just calling a service's API directly?	MCP Servers provide tool schemas + functions. If you want to directly call an API directly, you'll be authoring those on your own.
Sounds like MCP Servers and tool use are the same thing.	MCP Servers provide tool schemas + functions already defined for you.

ANTHROPIC

Who Authors MCP Servers?

Anyone can create an MCP server implementation. Often, service providers themselves will make their own official MCP implementations. For example, AWS might release an official MCP server with tools for their various services.

How is MCP Different from Direct API Calls?

MCP servers provide tool schemas and functions already defined for you. If you call an API directly, you're responsible for authoring those tool definitions yourself. MCP saves you that implementation work.

Isn't MCP Just Tool Use?

This is a common misconception. MCP servers and tool use are complementary but different concepts. MCP is about who does the work of creating and maintaining the tools. With MCP, someone else has already written the tool functions and schemas for you - they're packaged inside the MCP server.

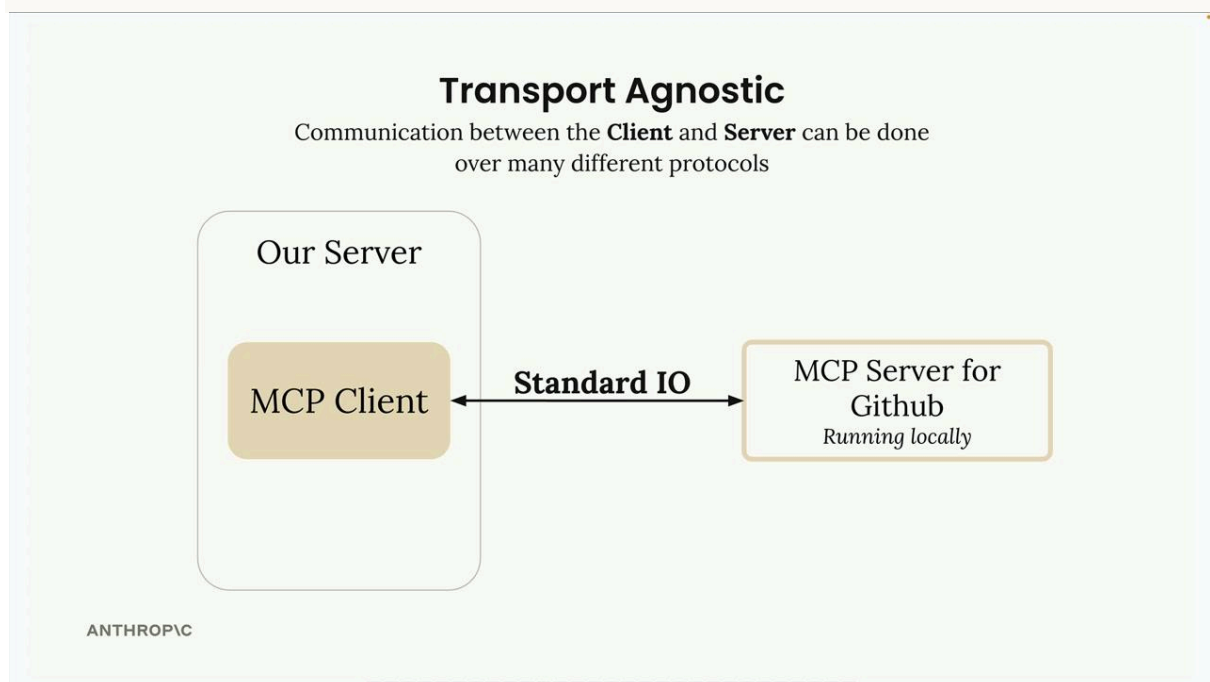
The key insight is that MCP servers provide tool schemas and functions already defined for you, eliminating the need to build and maintain complex integrations yourself.

MCP clients

The MCP client serves as the communication bridge between your server and MCP servers. Think of it as your access point to all the tools that an MCP server provides. When you need to use external tools or services, the client handles all the message passing and protocol details for you.

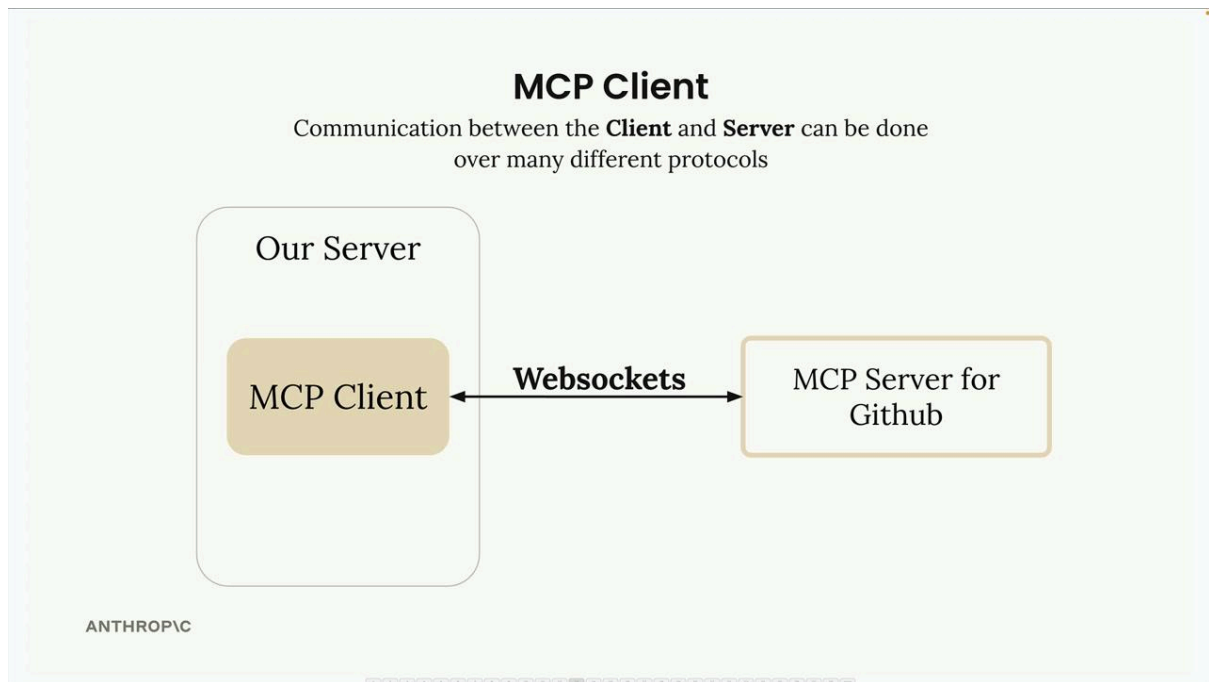
Transport Agnostic Communication

One of MCP's key strengths is being transport agnostic - a fancy way of saying the client and server can talk to each other using different communication methods. The most common setup runs both the MCP client and server on the same machine, where they communicate through standard input/output.



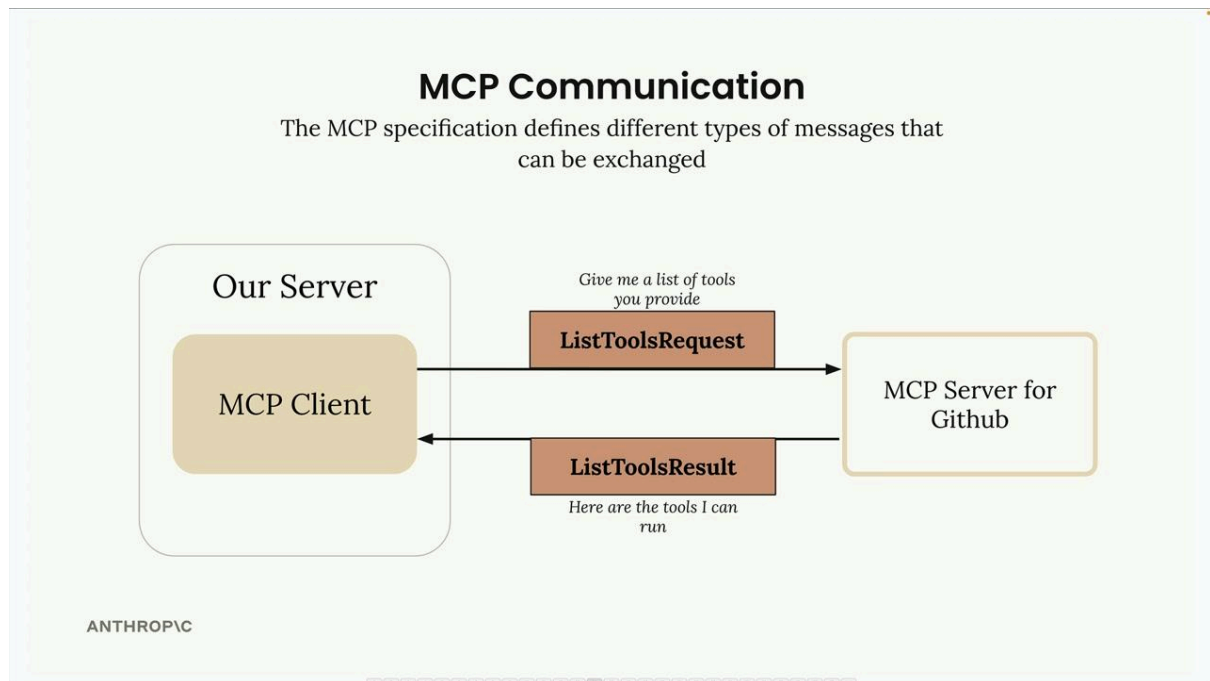
But you're not limited to that approach. MCP clients and servers can also connect over:

- HTTP
- WebSockets
- Various other network protocols

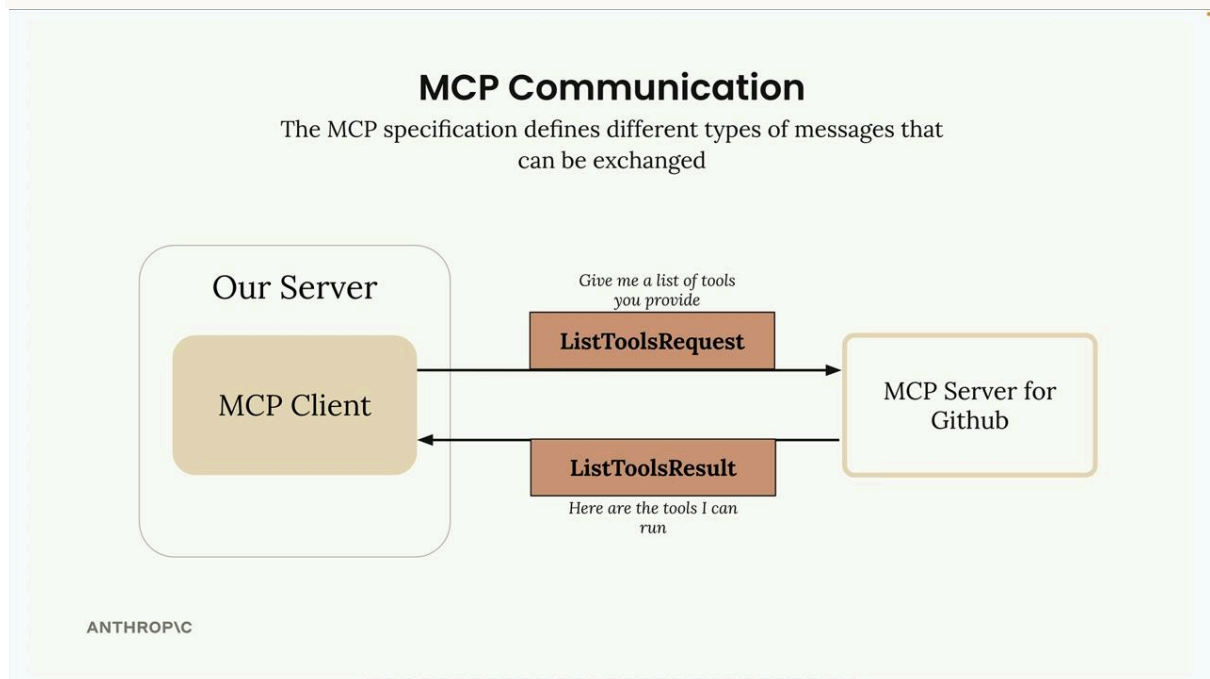


Message Types

Once connected, the client and server exchange specific message types defined in the MCP specification. The main message types you'll work with are:



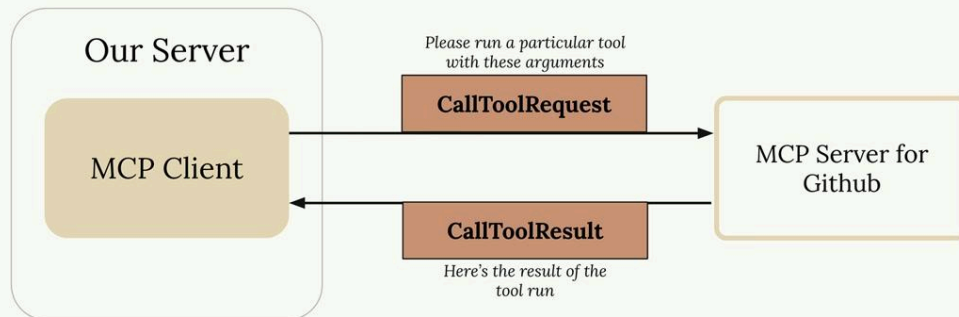
ListToolsRequest/ListToolsResult: The client asks the server "what tools do you provide?" and gets back a list of available tools.



CallToolRequest/CallToolResult: The client asks the server to run a specific tool with certain arguments, then receives the results.

MCP Communication

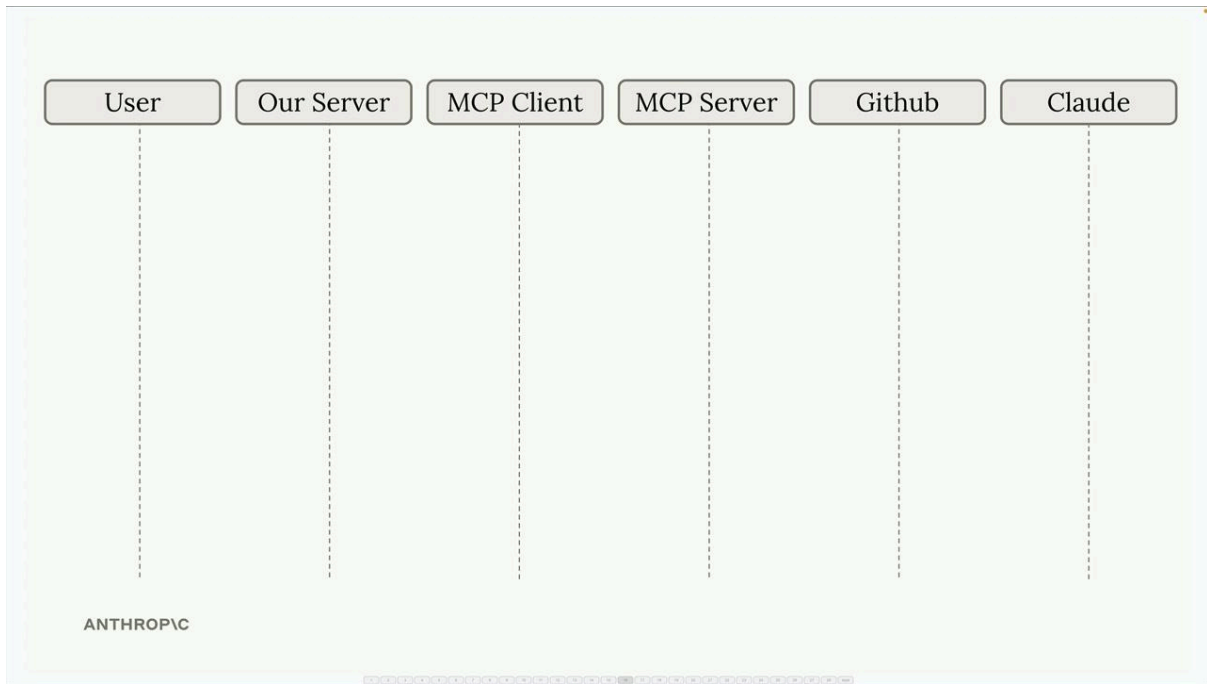
The MCP specification defines different types of messages that can be exchanged



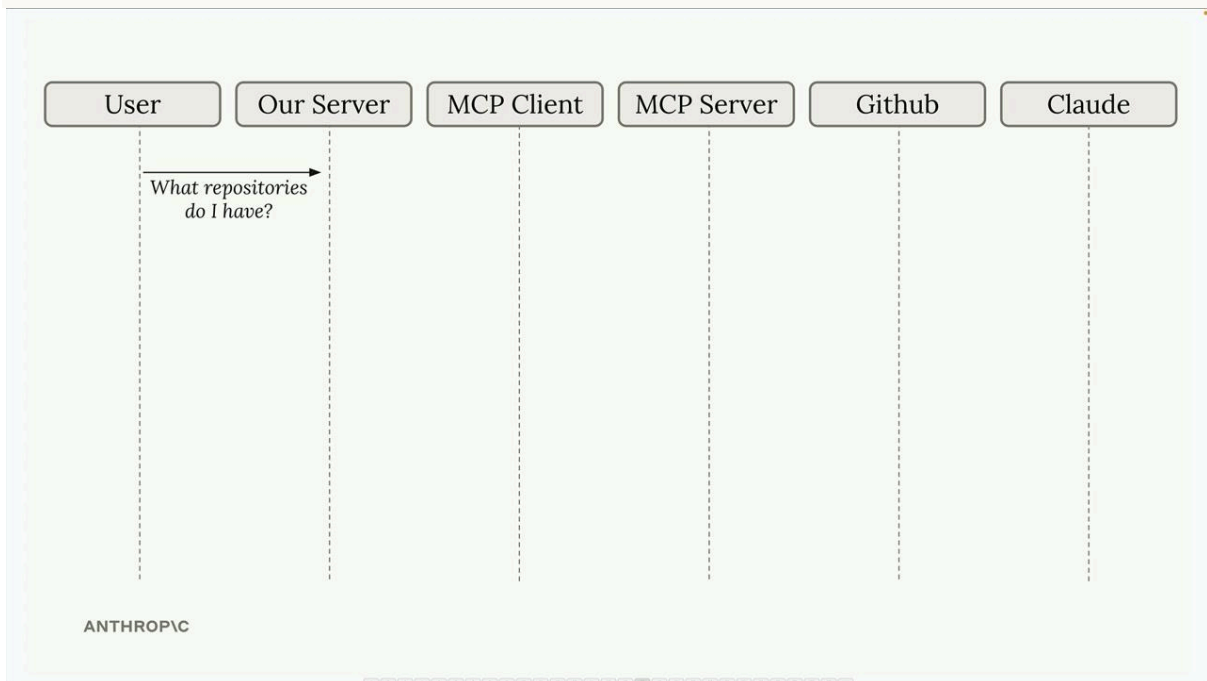
ANTHROPIC

Complete Flow Example

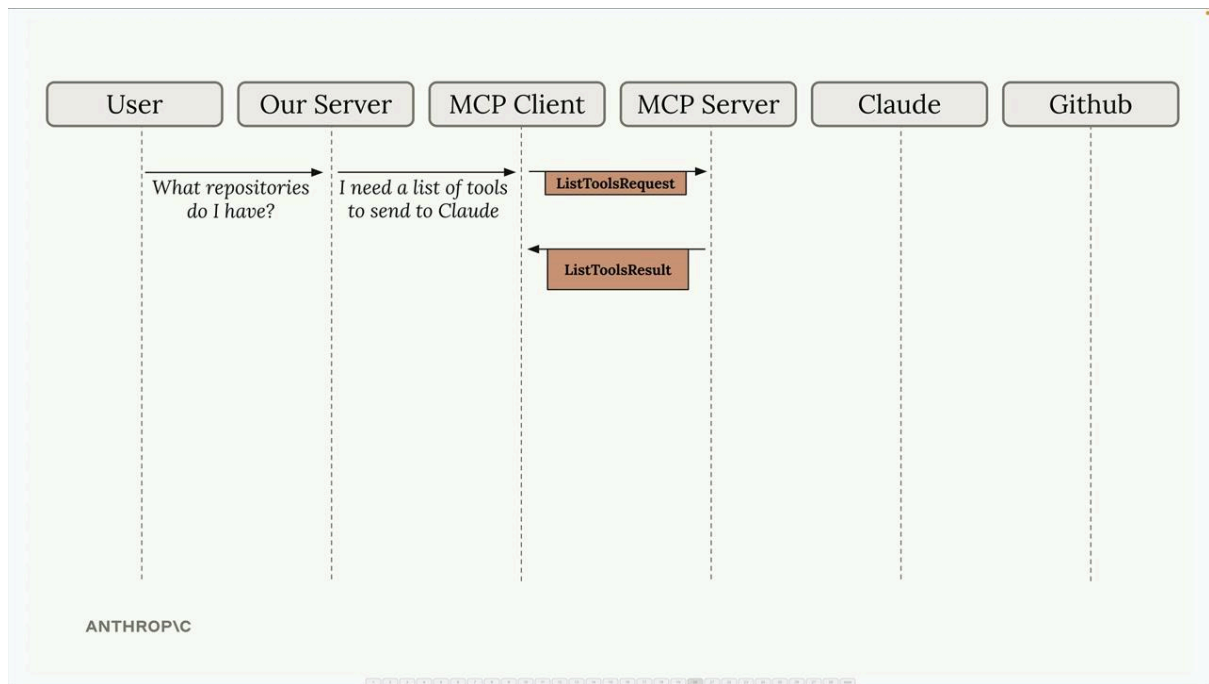
Here's how all the pieces work together in a real scenario. Let's say a user asks "What repositories do I have?" - here's the complete communication flow:



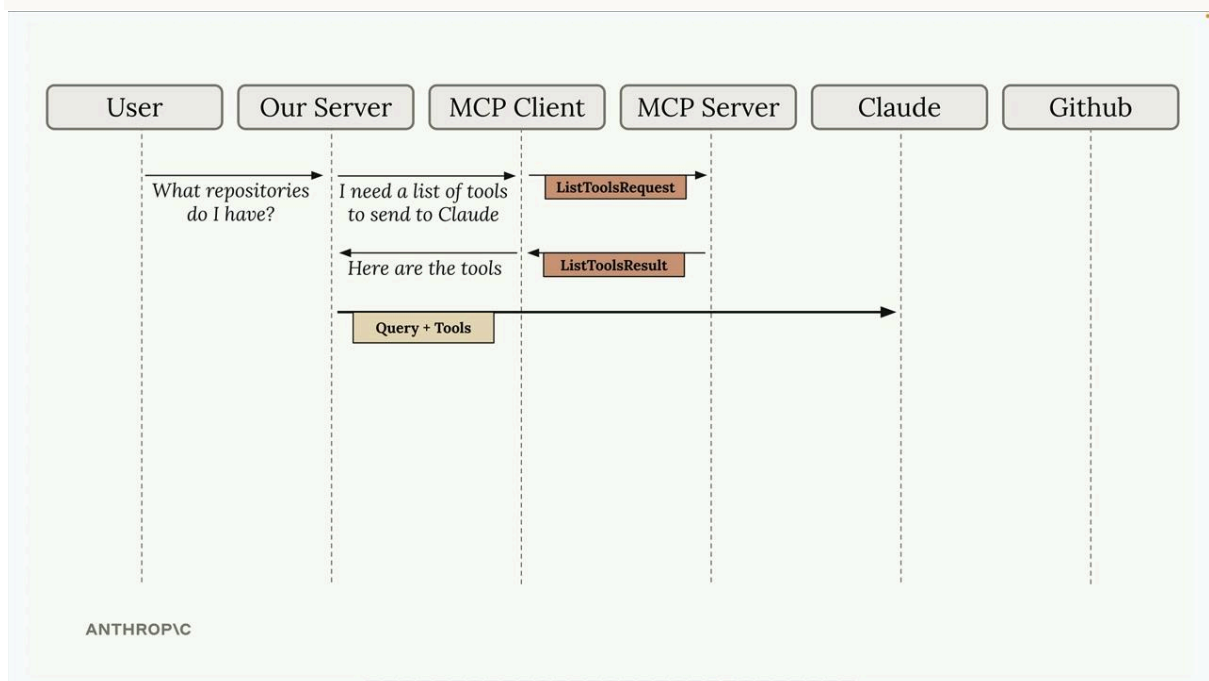
The process starts when a user submits a query to your server. Your server realizes it needs to provide Claude with a list of available tools before making the request.



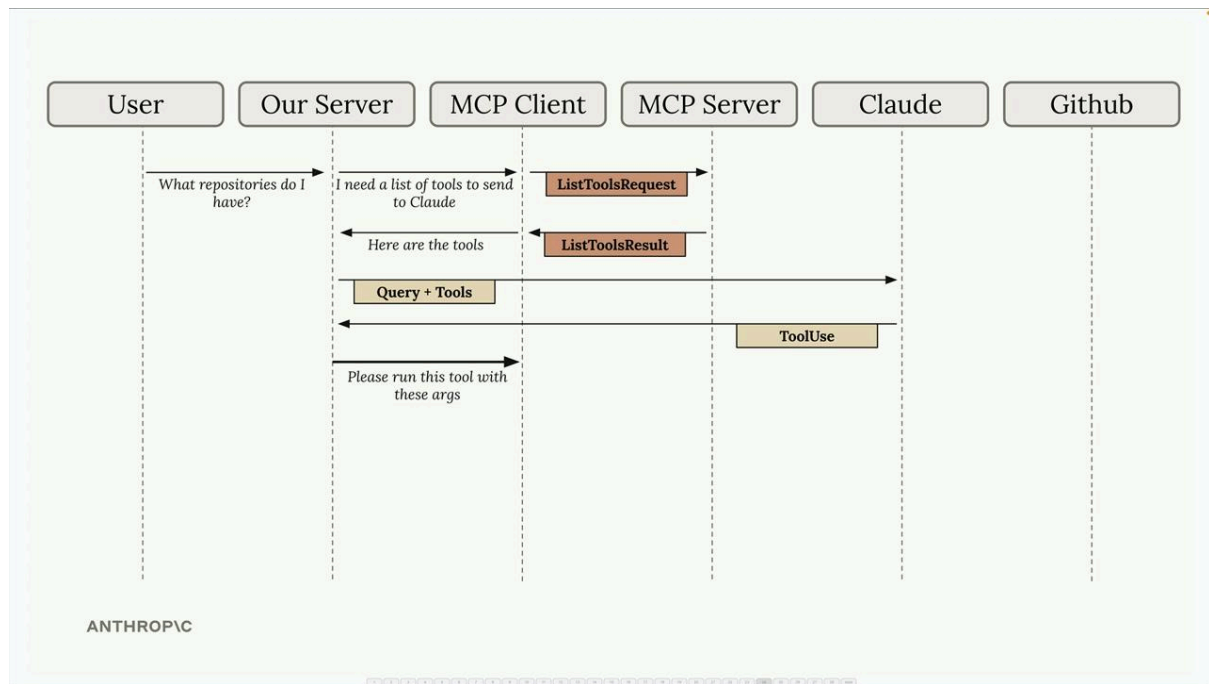
Your server asks the MCP client for tools, which sends a **ListToolsRequest** to the MCP server and receives a **ListToolsResult** back.



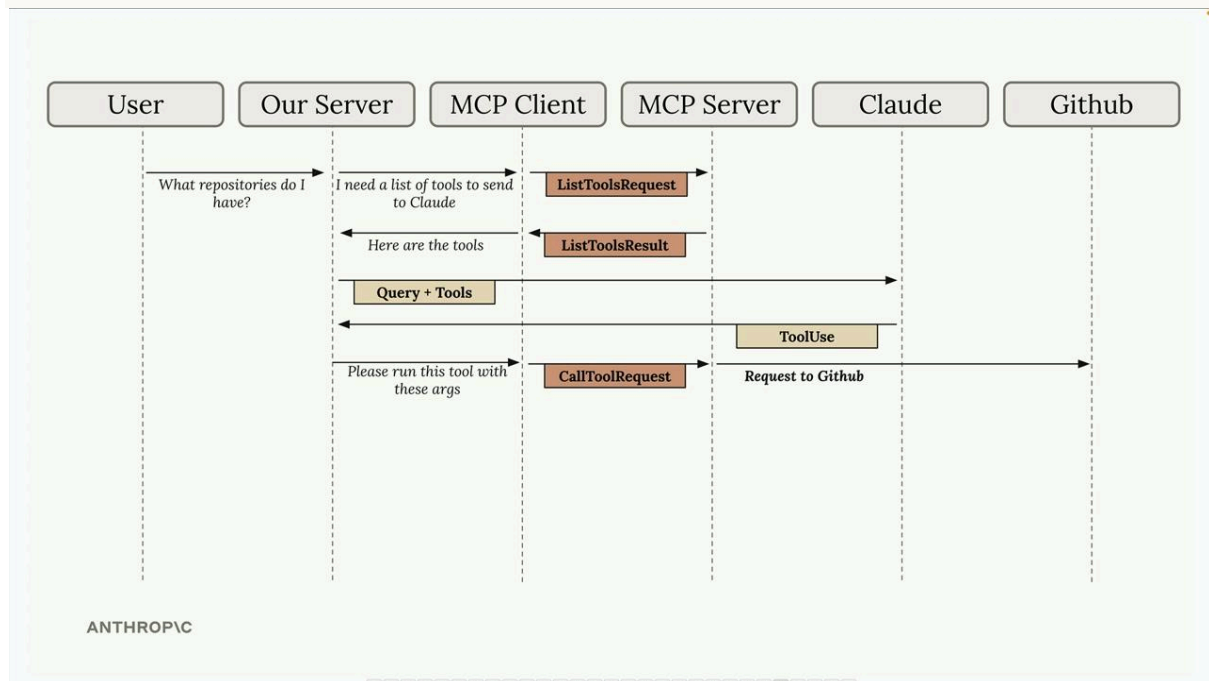
Now your server has everything needed to make the initial request to Claude - both the user's question and the available tools.



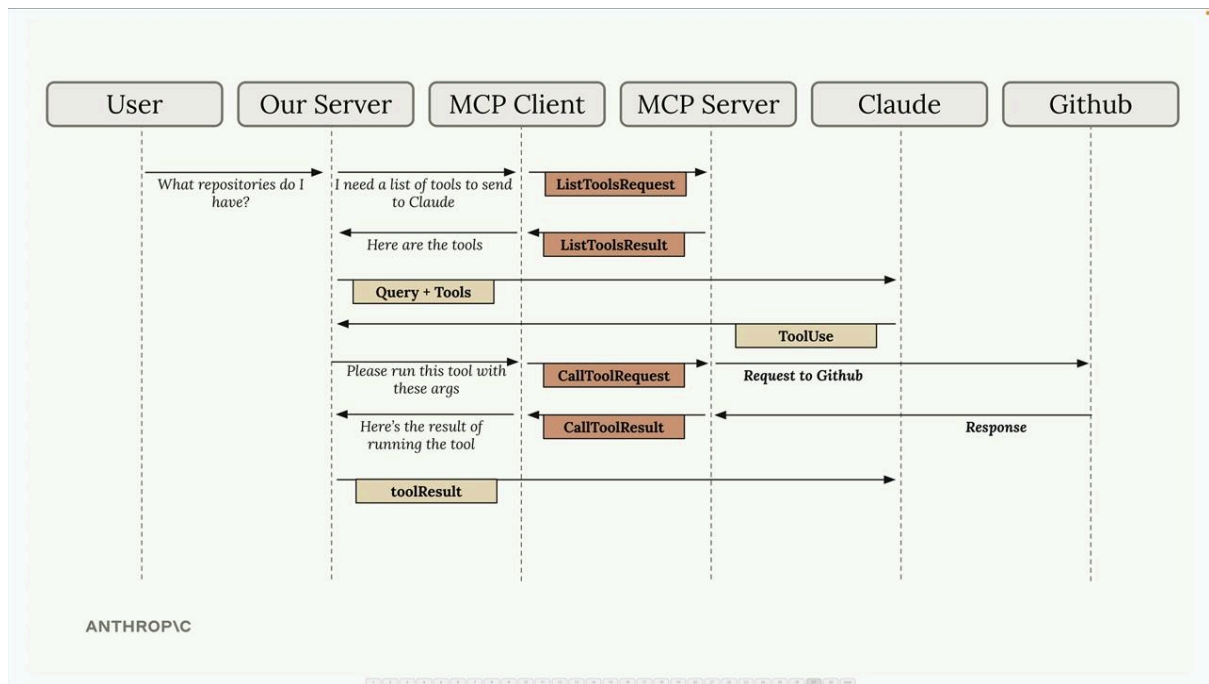
Claude examines the tools and decides it needs to call one to answer the question. It responds with a tool use request.



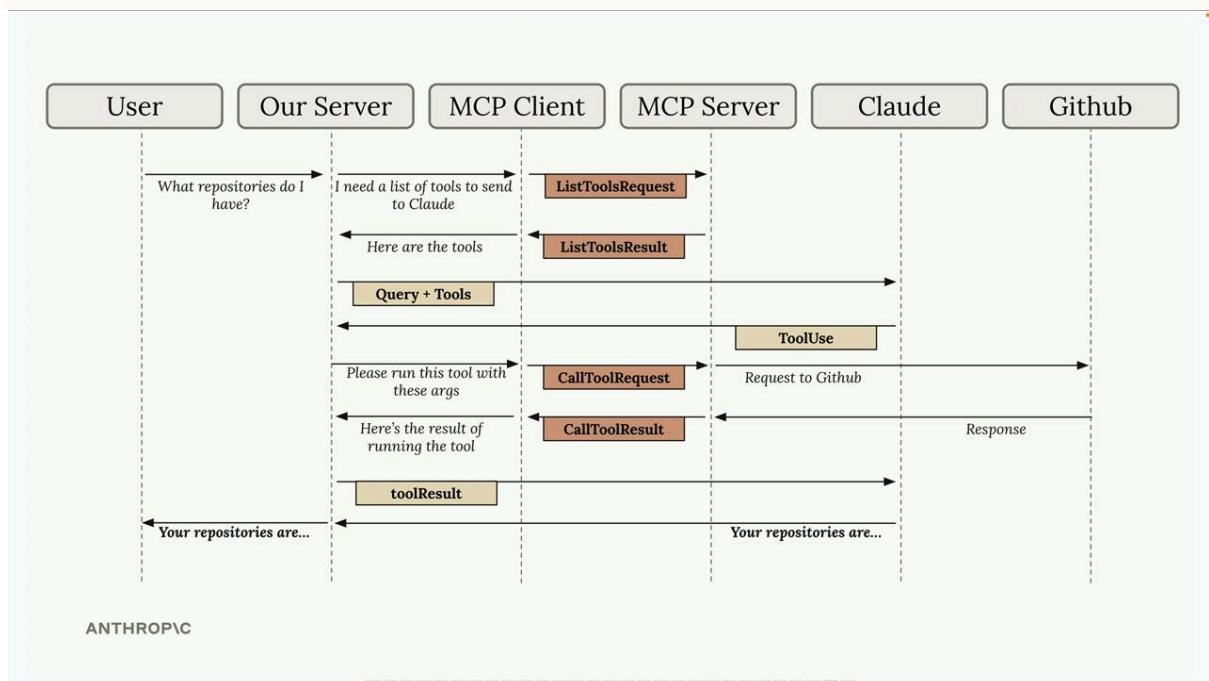
Your server asks the MCP client to execute the tool Claude requested. The MCP client sends a **CallToolRequest** to the MCP server, which then makes the actual request to GitHub.



GitHub returns the repository data, which flows back through the MCP server as a **CallToolResult**, then to the MCP client, and finally to your server.



Your server sends the tool results back to Claude in a follow-up message. Claude now has all the information it needs to formulate a complete response.



Finally, Claude responds with the formatted answer, which your server passes back to the user.

Yes, this flow involves many steps, but each component has a clear responsibility. The MCP client abstracts away the complexity of server communication, letting you focus on building your application logic. As we implement our own MCP client and server, you'll see how each piece fits together in practice.

Project setup

Downloads

`cli_project.zip` (opens in new tab)

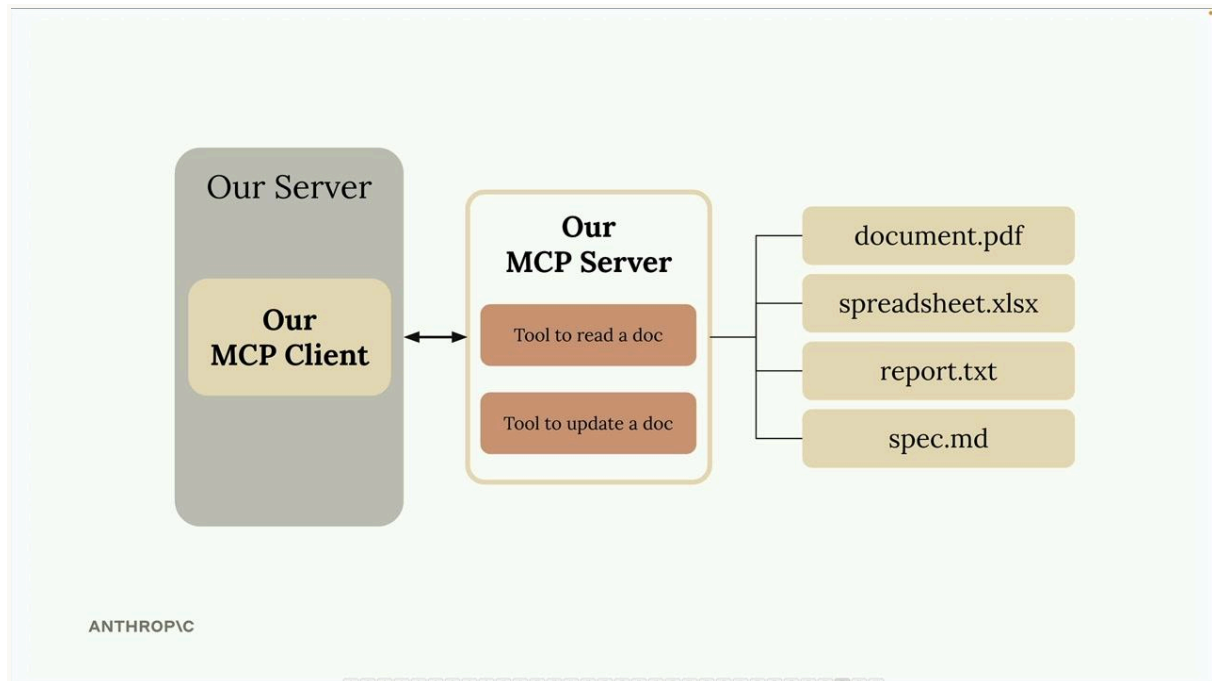
`cli_project_COMPLETE.zip`

We're going to build a CLI-based chatbot to better understand how MCP clients and servers work together. This hands-on project will give you practical experience with both sides of the MCP architecture.

What We're Building

Our chatbot will allow users to interact with a collection of documents through a command-line interface. The system consists of two main components:

- An MCP client that handles user interactions
- A custom MCP server that manages document operations



The server will provide two essential tools: one for reading document contents and another for updating them. All documents will be stored in memory for simplicity - no database required.

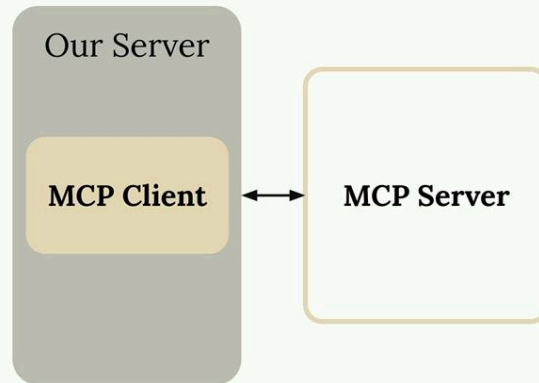
Important Architecture Note

In real-world projects, you typically implement either an MCP client or an MCP server, not both. You might create:

- An MCP server to expose your service to other developers
- An MCP client to connect to existing MCP servers

Important Note!

Normally a project will implement **either** an MCP client **or** an MCP server.
Our project will implement **both** just so we understand how they work



ANTHROPIC

We're building both components in this project purely for educational purposes - to understand how they communicate and work together.

Project Setup

Download the **cli_project.zip** file attached to this lesson and extract it to your preferred development directory. Open your code editor in the project folder.

The project includes a comprehensive README file with setup instructions. Follow these steps:

1. Add your Anthropic API key to the **.env** file
2. Install dependencies using either UV (recommended) or pip
3. Run the starter application to verify everything works

Running the Application

Navigate to your project directory in the terminal. You'll see the main project files including `main.py`, `mcp_client.py`, and `mcp_server.py`.

To start the application, use one of these commands:

```
# If using UV (recommended)
```

```
uv run main.py
```

```
# If using standard Python
```

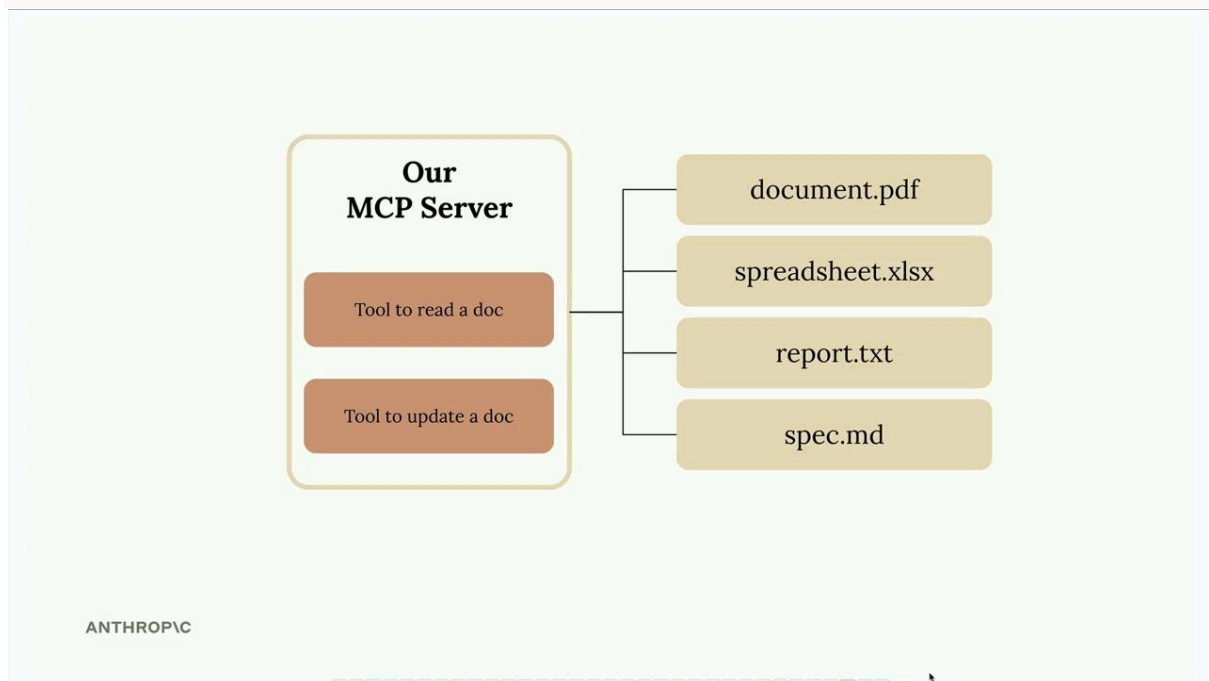
```
python main.py
```

When the application starts successfully, you'll see a chat prompt. Test it by asking a simple question like "what's 1+1?" - you should get a quick response from Claude.

With the basic setup complete, we're ready to start implementing MCP features and exploring how clients and servers communicate through the Model Control Protocol.

Defining tools with MCP

Building an MCP server becomes much simpler when you use the official Python SDK. Instead of manually writing complex JSON schemas for tools, the SDK handles all that complexity for you with decorators and type hints.



In this example, we're creating an MCP server that manages documents stored in memory. The server will provide two essential tools: one to read document contents and another to update them through find-and-replace operations.

Setting Up the MCP Server

The Python MCP SDK makes server creation incredibly straightforward. You can initialize a complete MCP server with just one line:

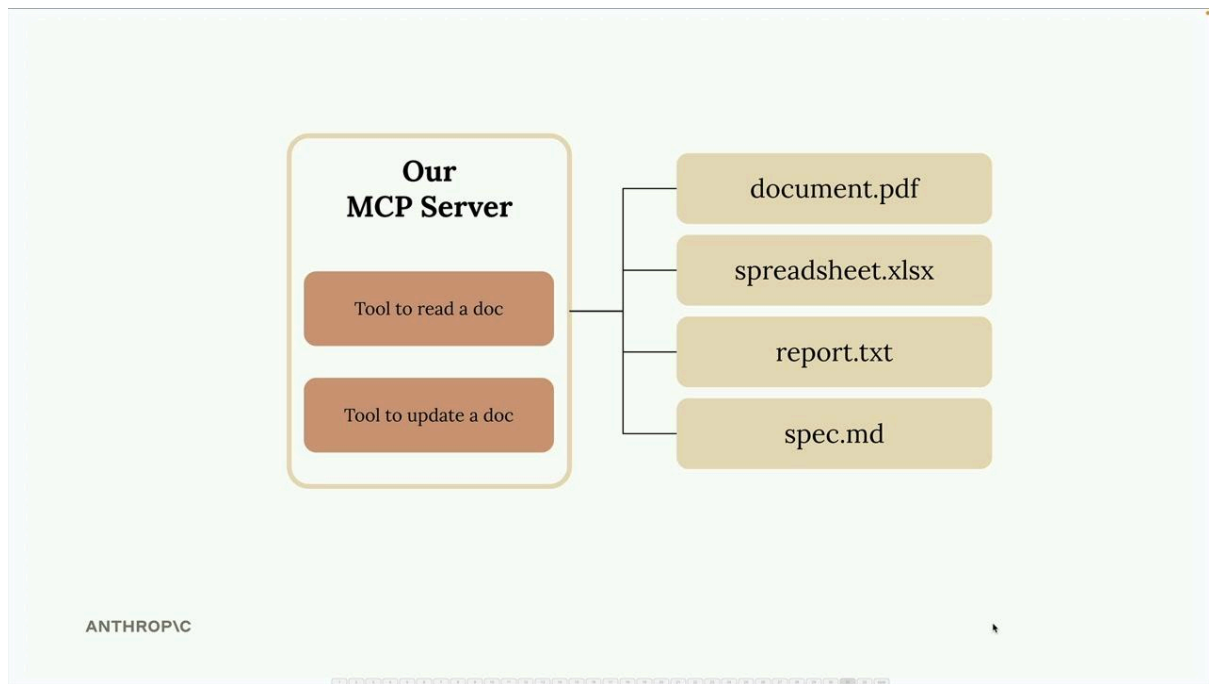
```
from mcp.server.fastmcp import FastMCP
```

```
mcp = FastMCP("DocumentMCP", log_level="ERROR")
```

For this implementation, documents are stored in a simple Python dictionary where keys are document IDs and values contain the document content:

```
docs = {  
    "deposition.md": "This deposition covers the testimony of  
Angela Smith, P.E.",  
    "report.pdf": "The report details the state of a 20m  
condenser tower.",  
    "financials.docx": "These financials outline the project's  
budget and expenditure",  
    "outlook.pdf": "This document presents the projected  
future performance of the",  
    "plan.md": "The plan outlines the steps for the project's  
implementation.",  
    "spec.txt": "These specifications define the technical  
requirements for the equipment"  
}
```

Tool Definition with Decorators



The SDK transforms tool creation from a verbose process into something clean and readable. Instead of writing lengthy JSON schemas, you use Python decorators and type hints.

Creating the Document Reader Tool

The first tool allows Claude to read any document by its ID. Here's the complete implementation:

```
@mcp.tool(  
    name="read_doc_contents",  
    description="Read the contents of a document and return it  
as a string."  
)  
def read_document(  
    doc_id: str = Field(description="Id of the document to  
read")  
):  
    if doc_id not in docs:  
        raise ValueError(f"Doc with id {doc_id} not found")
```

```
return docs[doc_id]
```

The `@mcp.tool` decorator automatically generates the JSON schema that Claude needs. The `Field` class from Pydantic provides parameter descriptions that help Claude understand what each argument expects.

Building the Document Editor Tool

The second tool performs simple find-and-replace operations on documents:

```
@mcp.tool(
    name="edit_document",
    description="Edit a document by replacing a string in the
documents content with a new string."
)
def edit_document(
    doc_id: str = Field(description="Id of the document that
will be edited"),
    old_str: str = Field(description="The text to replace.
Must match exactly, including whitespace."),
    new_str: str = Field(description="The new text to insert
in place of the old text.")
):
    if doc_id not in docs:
        raise ValueError(f"Doc with id {doc_id} not found")

    docs[doc_id] = docs[doc_id].replace(old_str, new_str)
```

This tool takes three parameters: the document ID, the text to find, and the replacement text. The implementation uses Python's built-in string `replace()` method for simplicity.

Error Handling

Both tools include basic error handling to manage cases where Claude requests a document that doesn't exist. When an invalid document ID is provided, the tools raise a **ValueError** with a descriptive message that Claude can understand and potentially act upon.

Key Benefits of the SDK Approach

- Automatic JSON schema generation from Python type hints
- Clean, readable code that's easy to maintain
- Built-in parameter validation through Pydantic
- Reduced boilerplate compared to manual schema writing
- Type safety and IDE support for development

The MCP Python SDK transforms what used to be a complex process of writing tool definitions into something that feels natural for Python developers. You focus on the business logic while the SDK handles the protocol details.

The server inspector

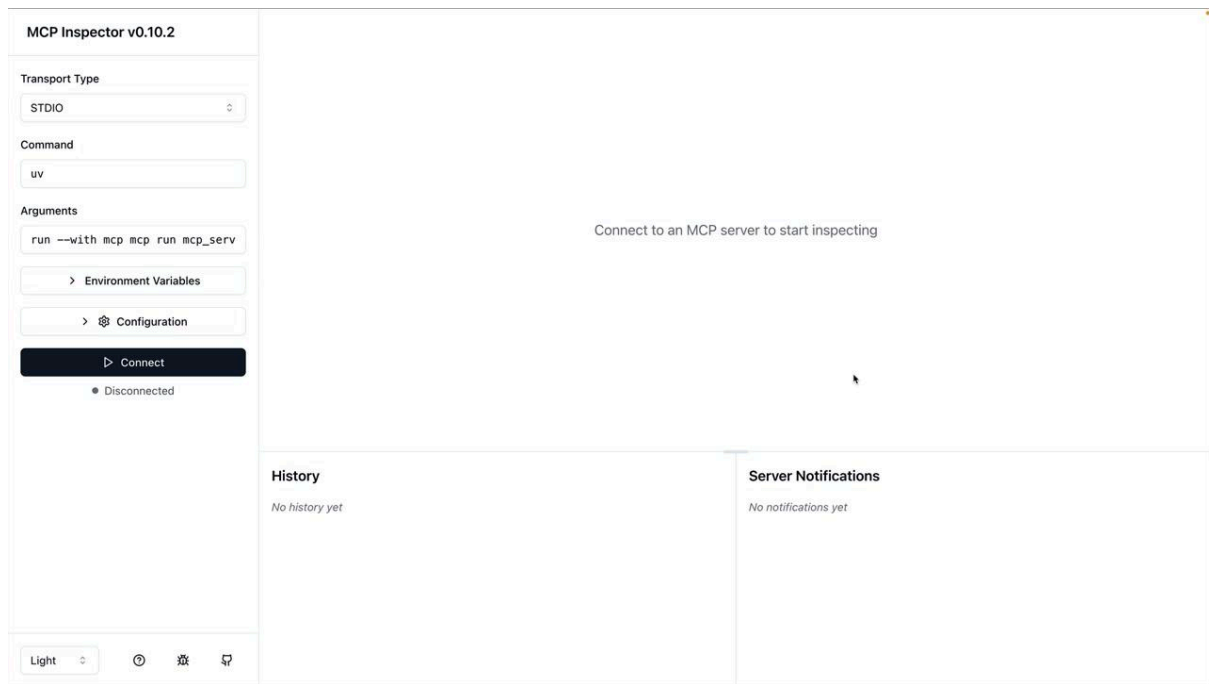
When building MCP servers, you need a way to test your functionality without connecting to a full application. The Python MCP SDK includes a built-in browser-based inspector that lets you debug and test your server in real-time.

Starting the Inspector

First, make sure your Python environment is activated (check your project's README for the exact command). Then run the inspector with:

```
mcp dev mcp_server.py
```

This starts a development server on port 6277 and gives you a local URL to open in your browser. The inspector interface will load, showing the MCP Inspector dashboard.

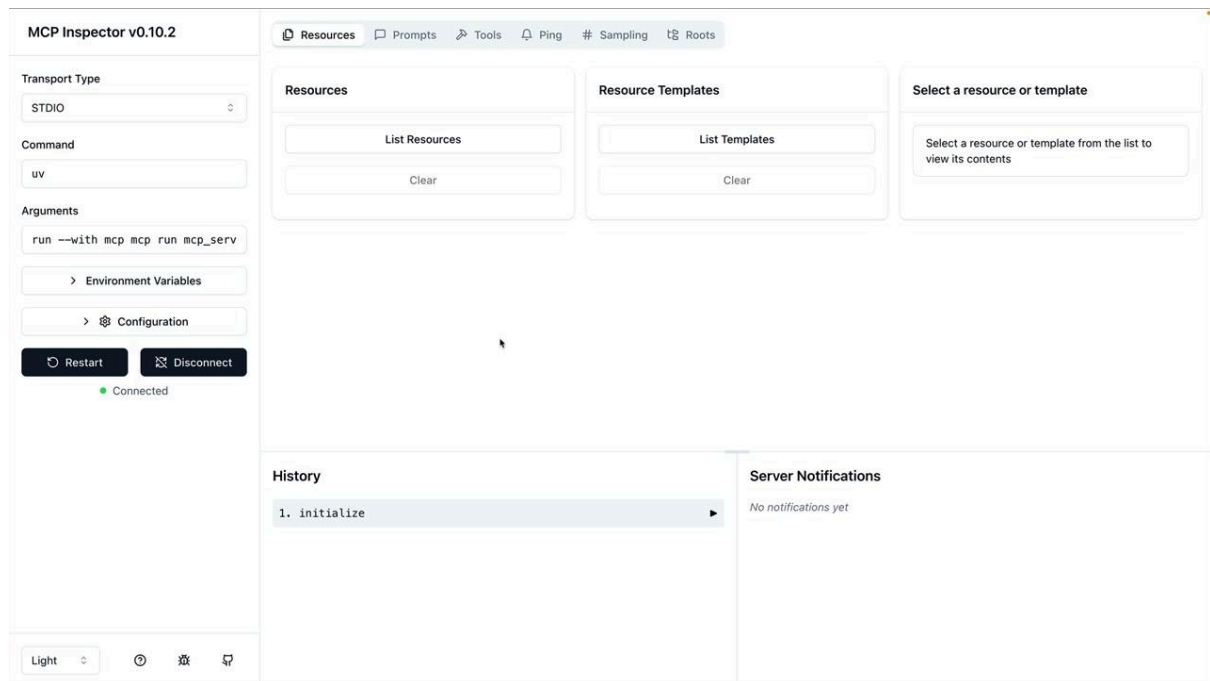


Important Note About the Interface

The MCP inspector is actively being developed, so the interface you see might look different from current screenshots. However, the core functionality for testing tools, resources, and prompts should remain similar.

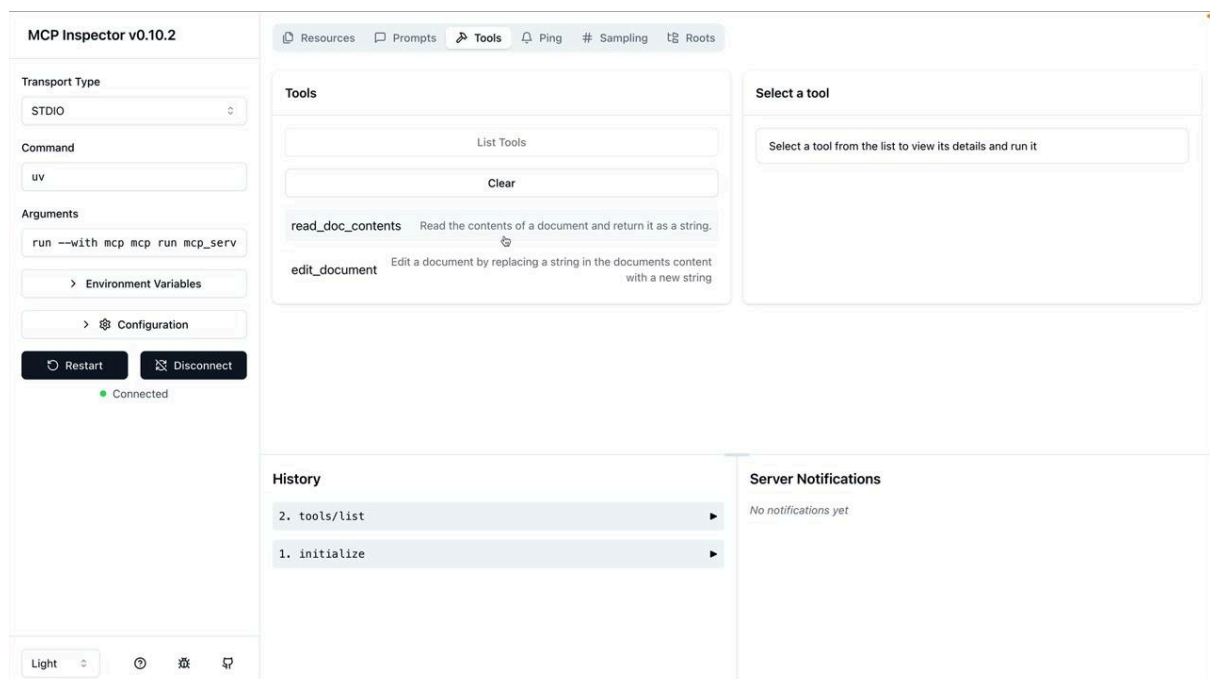
Connecting and Testing Tools

Click the "Connect" button on the left side to start your MCP server. Once connected, you'll see a navigation bar with sections for Resources, Prompts, Tools, and other features.



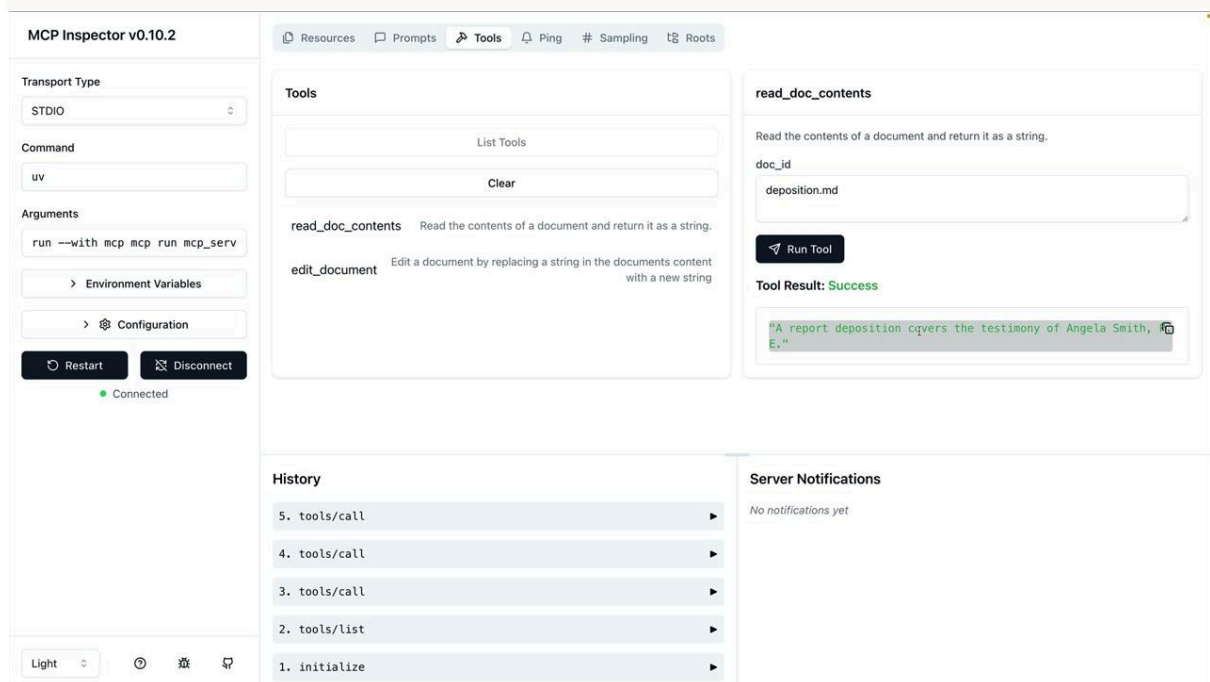
To test your tools:

- Navigate to the Tools section
- Click "List Tools" to see all available tools
- Select a tool to open its testing interface
- Fill in the required parameters
- Click "Run Tool" to execute and see results



Testing Document Operations

For example, to test a document reading tool, you'd enter a document ID (like "deposition.md") and run the tool. The inspector shows the result, including any returned content or success messages.



You can chain operations to verify functionality. For instance, after editing a document by replacing text, you can immediately run the read tool again to confirm the changes were applied correctly.

Development Workflow

The inspector creates an efficient development loop:

- Make changes to your MCP server code

- Test individual tools through the inspector
- Verify results without needing a full application setup
- Debug issues in isolation

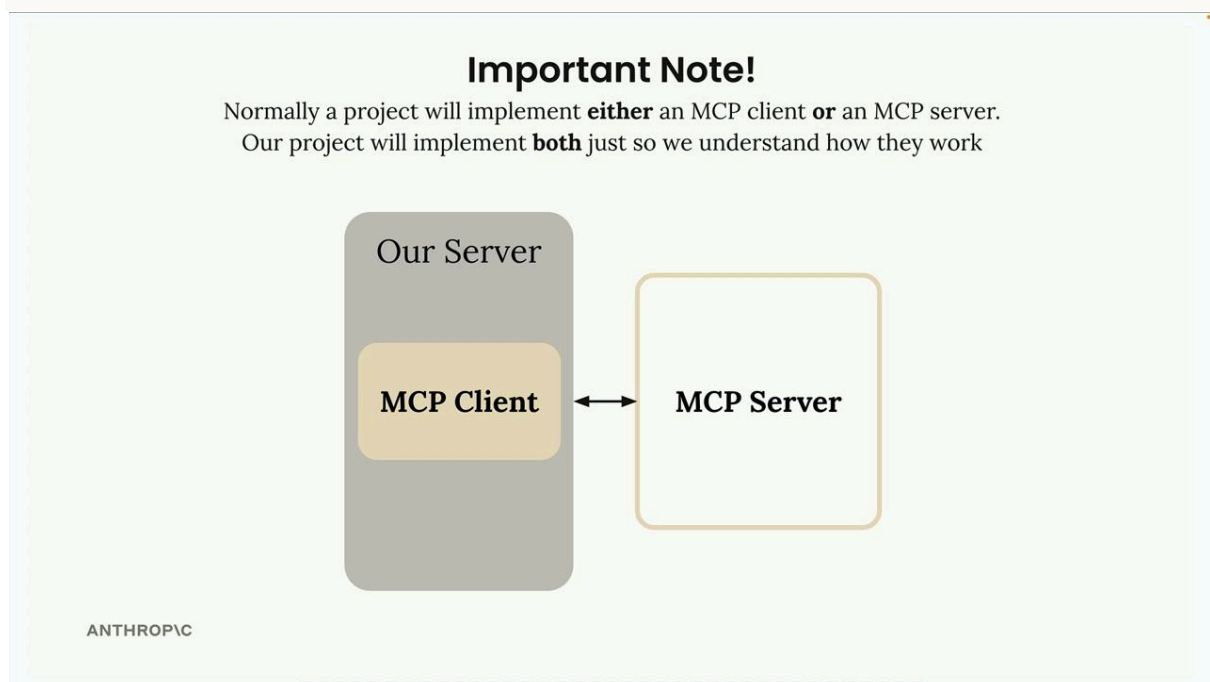
This tool becomes essential as you build more complex MCP servers. It eliminates the need to wire up your server to Claude or another application just to test basic functionality, making development much faster and more focused.

Implementing a client

Now that we have our MCP server working, it's time to build the client side. The client is what allows our application to communicate with the MCP server and access its functionality.

Understanding the Client Architecture

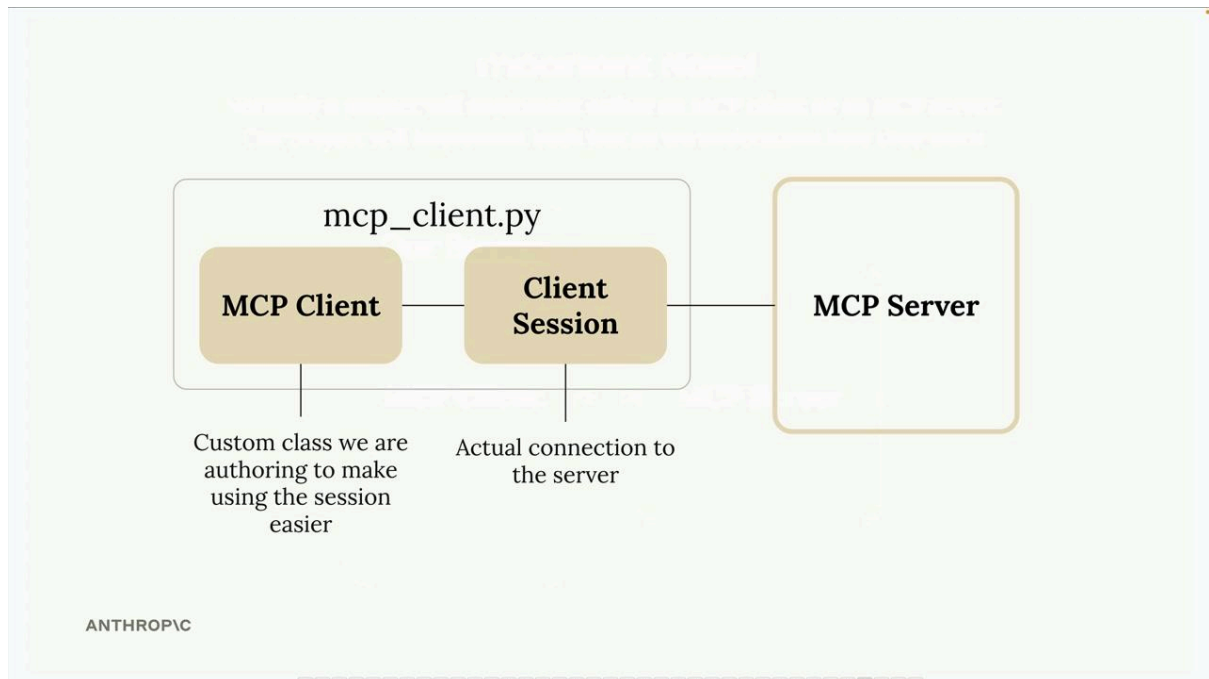
In most real-world projects, you'll either implement an MCP client OR an MCP server - not both. We're building both in this project just so you can see how they work together.



The MCP client consists of two main components:

- MCP Client - A custom class we create to make using the session easier

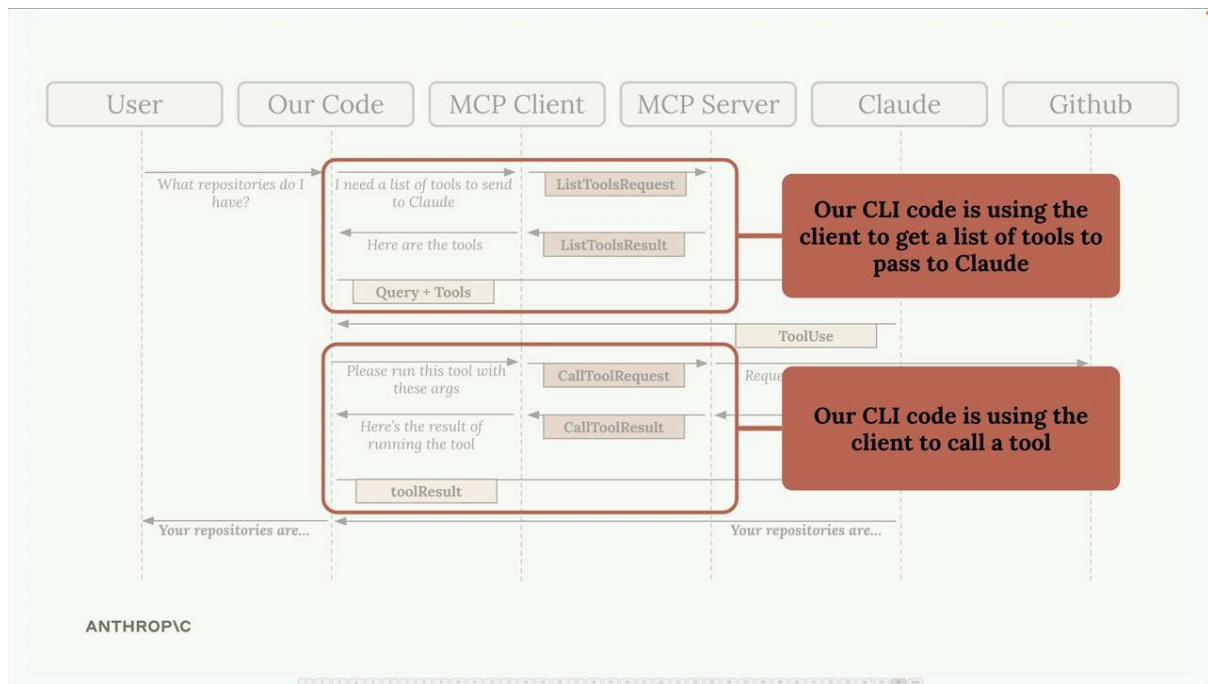
- Client Session - The actual connection to the server (part of the MCP Python SDK)



The client session requires proper resource cleanup when we're done with it. That's why we wrap it in our custom MCP Client class - to handle all that cleanup automatically.

How the Client Fits Into Our Application

Remember our application flow? Our CLI code needs to do two main things with the MCP server:



1. Get a list of available tools to send to Claude
2. Execute tools when Claude requests them

The MCP client provides these capabilities through simple method calls that our application code can use.

Implementing the Core Methods

We need to implement two key methods in our client: `list_tools()` and `call_tool()`.

List Tools Method

This method gets all available tools from the server:

```
async def list_tools(self) -> list[types.Tool]:
    result = await self.session().list_tools()
```

```
return result.tools
```

It's straightforward - we access our session (the connection to the server), call the built-in `list_tools()` function, and return the tools from the result.

Call Tool Method

This method executes a specific tool on the server:

```
async def call_tool(
    self, tool_name: str, tool_input: dict
) -> types.CallToolResult | None:

    return await self.session().call_tool(tool_name,
tool_input)
```

We pass the tool name and input parameters (provided by Claude) to the server and return the result.

Testing the Client

To test our implementation, we can run the client directly. The file includes a testing harness that connects to our MCP server and calls our methods:

```
async with MCPClient(
    command="uv", args=["run", "mcp_server.py"]
) as client:
    result = await client.list_tools()

    print(result)
```

When we run this test, we should see our tool definitions printed out, including the **read_doc_contents** and **edit_document** tools we created earlier.

Putting It All Together

Now that our client can list tools and call them, we can test the complete flow. When we run our main application and ask Claude about a document:

1. Our code uses the client to get available tools
2. These tools are sent to Claude along with the user's question
3. Claude decides to use the **read_doc_contents** tool
4. Our code uses the client to execute that tool
5. The result is sent back to Claude, who then responds to the user

For example, asking "What is the contents of the report.pdf document?" will trigger Claude to use our document reading tool, and we'll get back information about the 20m condenser tower document we set up in our server.

The client acts as the bridge between our application logic and the MCP server, making it easy to access server functionality without worrying about the underlying connection details.

Defining resources

Resources in MCP servers allow you to expose data to clients, similar to GET request handlers in a typical HTTP server. They're perfect for scenarios where you need to fetch information rather than perform actions.

Understanding Resources Through an Example

Let's say you want to build a document mention feature where users can type **@document_name** to reference files. This requires two operations:

- Getting a list of all available documents (for autocomplete)
- Fetching the contents of a specific document (when mentioned)

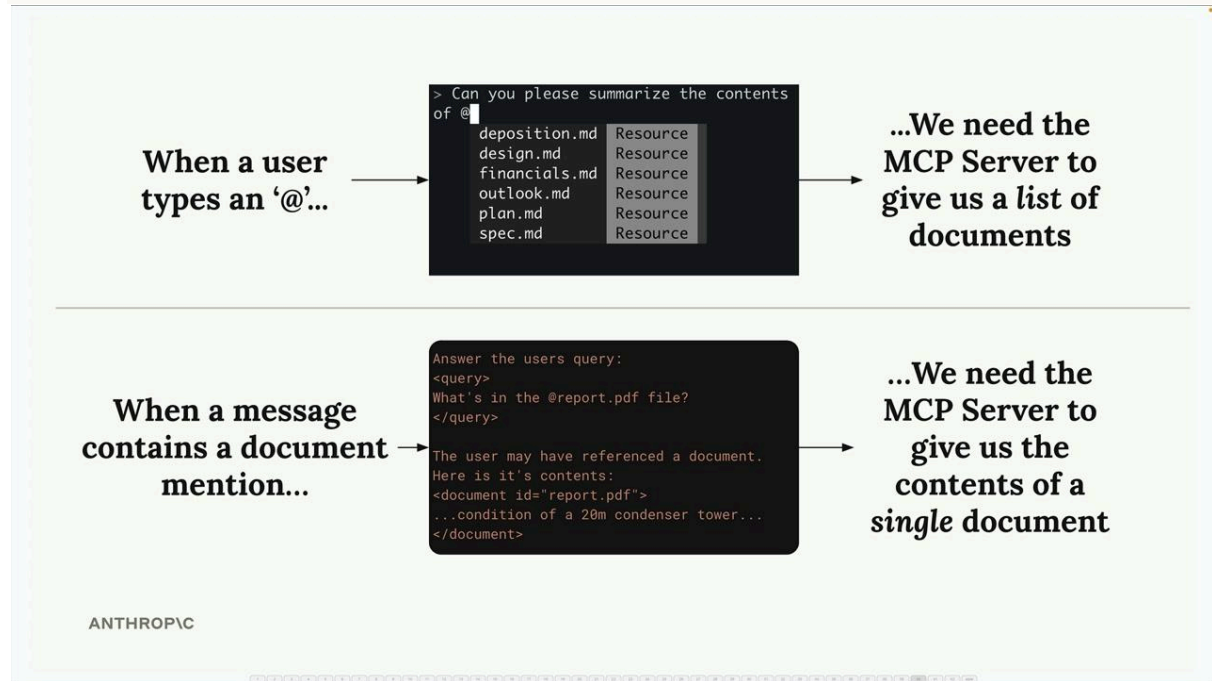
Next Feature

- Users can “mention” a document by writing out “@doc_name”
 - Typing “@” should show a list of all the available documents
 - When a document is mentioned, its contents should be automatically injected into the prompt

ANTHROPIC

```
> Can you please summarize the contents
of @
deposition.md Resource
design.md      Resource
financials.md Resource
outlook.md    Resource
plan.md       Resource
spec.md       Resource
```

When a user types @, you need to show available documents. When they submit a message with a mention, you automatically inject that document's content into the prompt sent to Claude.



How Resources Work

Resources follow a request-response pattern. Your client sends a **ReadResourceRequest** with a URI, and the MCP server responds with the data. The URI acts like an address for the resource you want to access.

Resources

- Allow the MCP Server to expose data to the client
- Similar to GET request handlers in a typical HTTP server
- Can return any type of data - strings, JSON, binary, etc.
 - We set the 'mime_type' to give the client a hint as to what data we are returning
- Two types: direct and templated

ANTHROPIC

MCP Server

Resource

```
@mcp.resource(
    "docs://documents",
    mime_type="application/json"
)
def list_docs():
    # Return a list of document names
```

Resource +

```
@mcp.resource(
    "docs://documents/{doc_id}",
    mime_type="text/plain"
)
def fetch_doc(doc_id: str):
    # Return the contents of a doc
```

Types of Resources

There are two types of resources:

```
@mcp.resource(
    "docs://documents", # URI
    mime_type="application/json"
)
def list_docs():
    # Return a list of document names
```

Direct Resource

URI doesn't contain any params.

```
@mcp.resource(
    "docs://documents/{doc_id}", # URI
    mime_type="text/plain"
)
def fetch_doc(doc_id: str):
    # Return the contents of a doc
```

Templated Resource

URI contains one or more params.
The Python SDK parses these and passes them as args to your function.

ANTHROPIC

- Direct Resources: Static URIs that don't change, like
`docs://documents`
- Templated Resources: URIs with parameters, like
`docs://{doc_id}`

For templated resources, the Python SDK automatically parses parameters from the URI and passes them as keyword arguments to your function.

Implementing Resources

Resources are defined using the `@mcp.resource()` decorator. Here's how to create both types:

Direct Resource (List Documents)

```
@mcp.resource(  
    "docs://documents",  
    mime_type="application/json"  
)  
def list_docs() -> list[str]:  
  
    return list(docs.keys())
```

Templated Resource (Fetch Document)

```
@mcp.resource(  
    "docs://{doc_id}",  
    mime_type="text/plain"  
)  
def fetch_doc(doc_id: str) -> str:  
    if doc_id not in docs:  
        raise ValueError(f"Doc with id {doc_id} not found")
```

```
return docs[doc_id]
```

MIME Types

Resources can return any type of data - strings, JSON, binary, etc. The **mime_type** parameter gives clients a hint about what kind of data you're returning:

- **application/json** - Structured JSON data
- **text/plain** - Plain text content
- Any other valid MIME type for different data formats

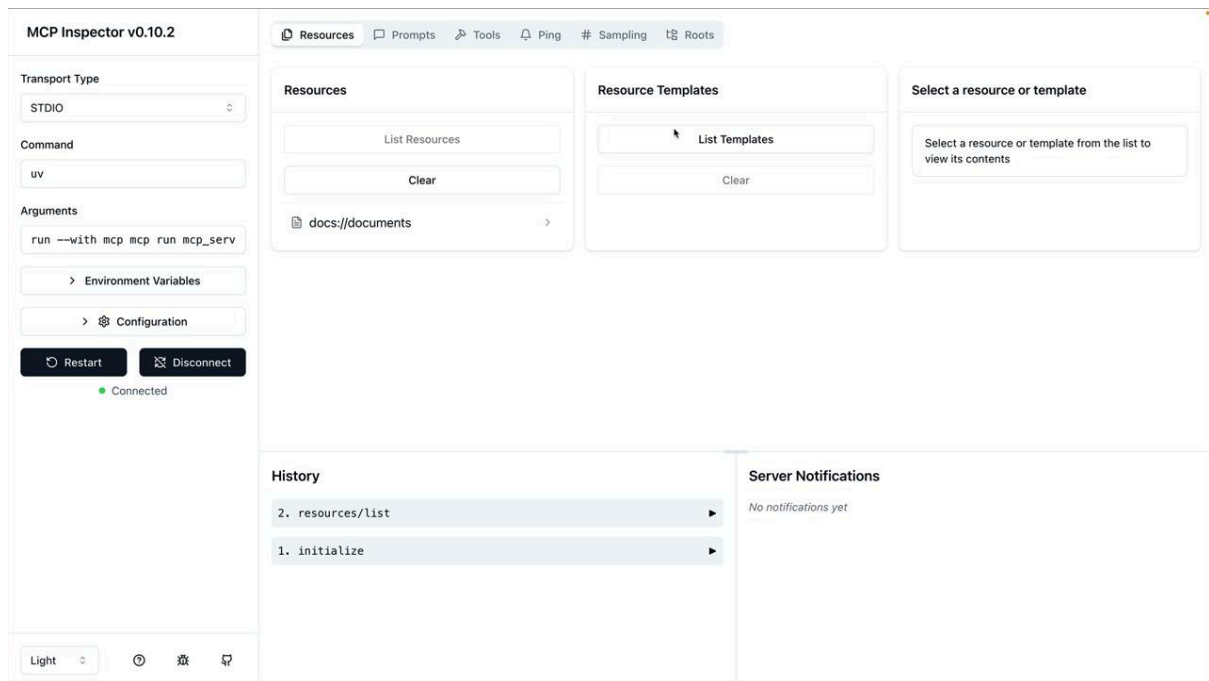
The MCP Python SDK automatically serializes your return values. You don't need to manually convert to JSON strings.

Testing Resources

You can test your resources using the MCP Inspector. Run your server with:

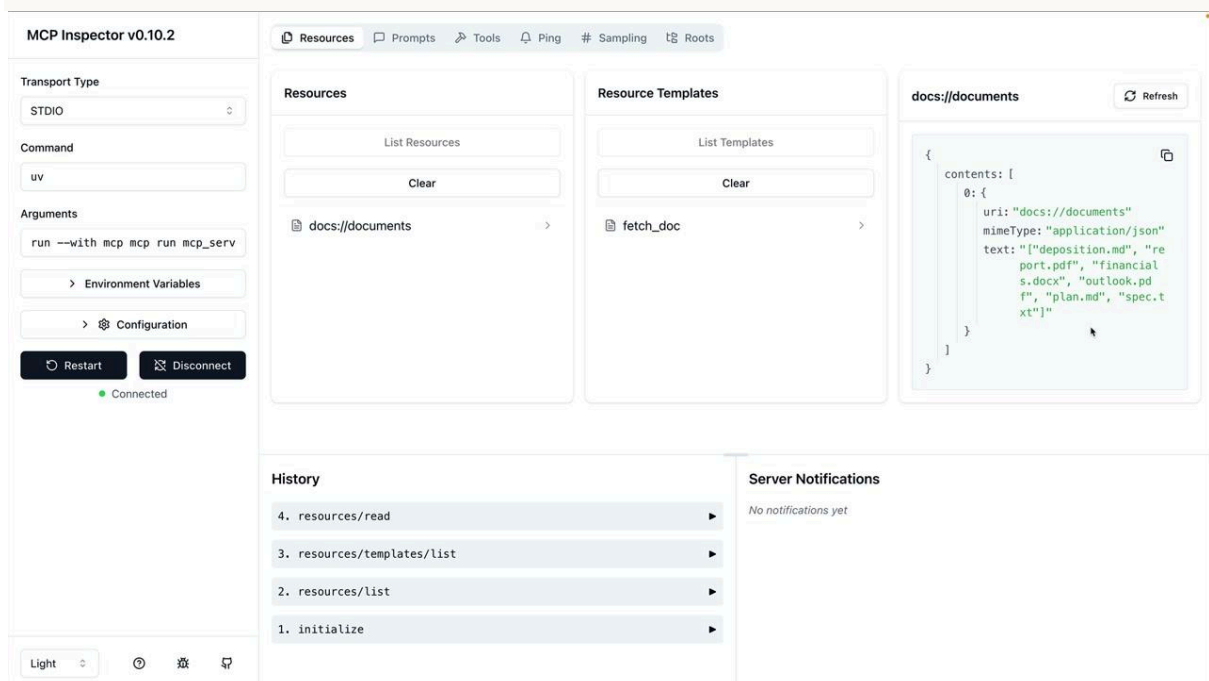
```
uv run mcp dev mcp_server.py
```

Then connect to the inspector in your browser. You'll see:



- Resources: Lists your direct/static resources
- Resource Templates: Shows templated resources that accept parameters

Click on any resource to test it and see the exact response structure your client will receive.



Key Points

- Resources expose data, tools perform actions
- Use direct resources for static data, templated resources for parameterized queries
- MIME types help clients understand response format
- The SDK handles serialization automatically
- Parameter names in templated URIs become function arguments

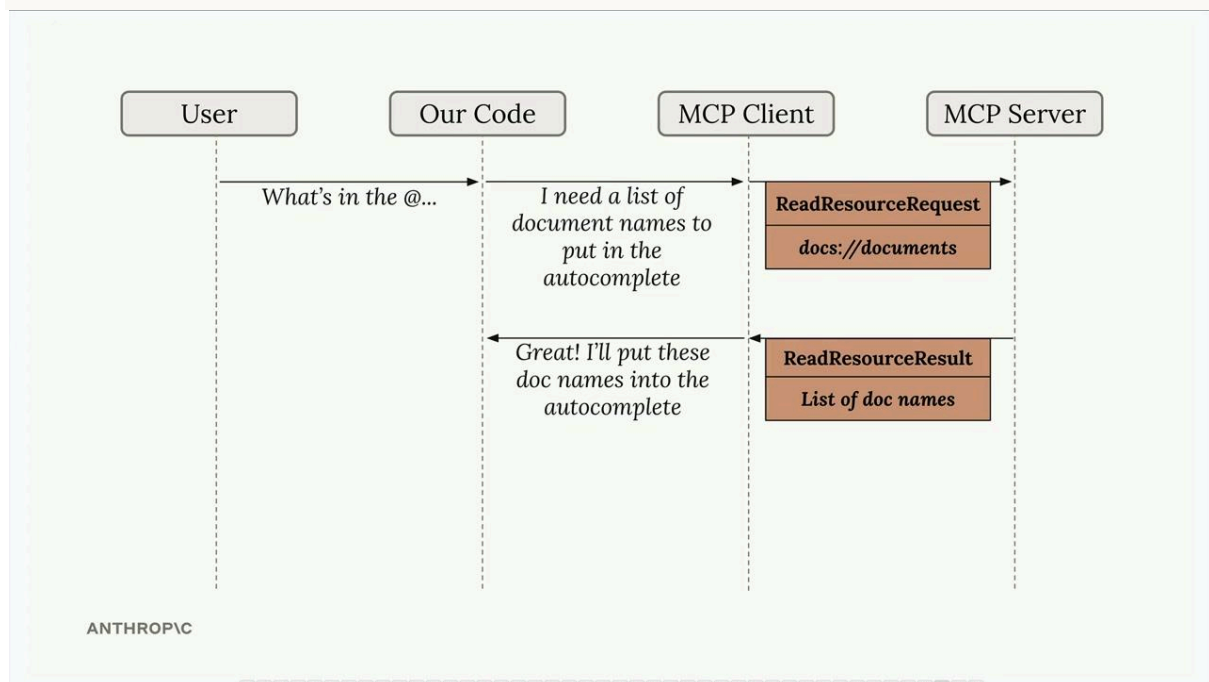
Resources provide a clean way to make data available to MCP clients, enabling features like document mentions, file browsing, or any scenario where you need to fetch information from your server.

Accessing resources

Resources in MCP allow your server to expose data that can be directly included in prompts, rather than requiring tool calls to access information. This creates a more efficient way to provide context to AI models like Claude.

Understanding Resource Requests

When you've defined resources on your MCP server, your client needs a way to request and use them. The client acts as a bridge between your application and the MCP server, handling the communication and data parsing automatically.



The flow is straightforward: when a user wants to reference a document (like typing "`@report.pdf`"), your application uses the MCP client to fetch

that resource from the server and include its contents directly in the prompt sent to Claude.

Implementing Resource Reading

The core functionality requires a **read_resource** function in your MCP client. This function takes a URI parameter identifying which resource to fetch:

```
async def read_resource(self, uri: str) -> Any:
    result = await self.session().read_resource(AnyUrl(uri))

    resource = result.contents[0]
```

The response from the MCP server contains a **contents** list. You typically only need the first element, which contains the actual resource data along with metadata like the MIME type.

Handling Different Content Types

Resources can return different types of content, so your client needs to parse them appropriately. The MIME type tells you how to handle the data:

```
if isinstance(resource, types.TextResourceContents):
    if resource.mimeType == "application/json":
        return json.loads(resource.text)
```

```
return resource.text
```

This approach ensures that JSON resources are properly parsed into Python objects, while plain text resources are returned as strings. The MIME type acts as your hint for determining the correct parsing strategy.

Required Imports

To make this work properly, you'll need these imports in your MCP client:

```
import json
```

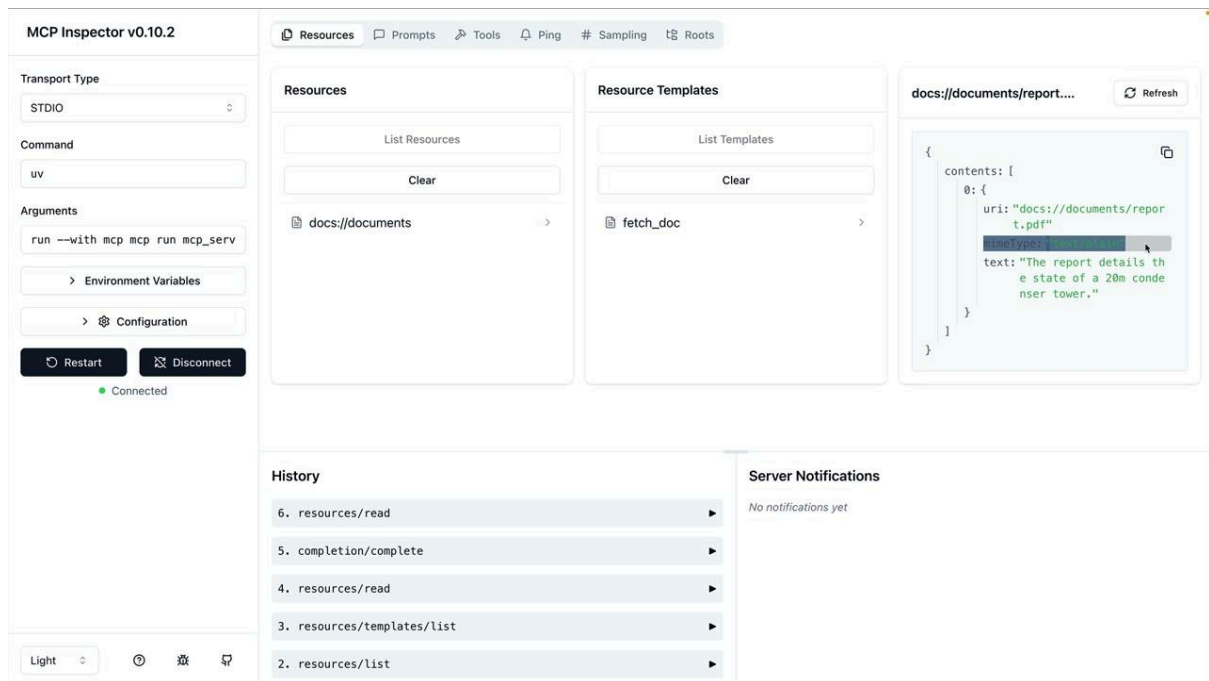
```
from pydantic import AnyUrl
```

The **json** module handles parsing JSON responses, while **AnyUrl** ensures proper type handling for the URI parameter.

Testing Resource Access

Once implemented, you can test the functionality through your CLI application. When you type something like "What's in the @report.pdf document?", the system should:

- Show available resources in an autocomplete list
- Allow you to select a resource
- Fetch the resource content automatically
- Include that content in the prompt to Claude



The key advantage is that Claude receives the document content directly in the prompt, eliminating the need for tool calls to access the information. This makes interactions faster and more efficient.

Integration with Your Application

Remember that the MCP client code you write gets used by other parts of your application. The **read_resource** function becomes a building block that other components can call to fetch document contents, list available resources, or integrate resource data into prompts.

This separation of concerns keeps your code clean: the MCP client handles communication with the server, while your application logic focuses on how to use that data effectively.

Defining prompts

Prompts in MCP servers let you define pre-built, high-quality instructions that clients can use instead of writing their own prompts from scratch. Think of them as carefully crafted templates that give better results than what users might come up with on their own.

Why Use Prompts?

Let's say you want Claude to reformat a document into markdown. A user could just type "convert report.pdf to markdown" and it would work fine. But they'd probably get much better results with a thoroughly tested prompt that includes specific instructions about formatting, structure, and output requirements.

If we left this process up to a user, here's what they'd write:

```
Convert report.pdf to markdown
```

Yes, it'd work, but the user might get a better result with some strong prompt engineering

ANTHROPIC

User might have more luck if they use our thoroughly-eval'd prompt instead!

```
You are a document conversion specialist tasked with rewriting documents in Markdown format. Your goal is to take the content of a given document and convert it into well-structured Markdown, preserving the original meaning and enhancing readability. Here is the identifier of the document you need to convert:
<document_id>
{{doc_id}}
</document_id>
Instructions:
1. Retrieve the content of the document associated with the given document_id.
2. Analyze the structure and content of the document.
3. Convert the document to Markdown format, following these guidelines:
- Use appropriate header levels (# for main titles, ## for subtitles, etc.)
- Properly format lists (both ordered and unordered)
- Use emphasis ("italics" or "bold") where appropriate
- Add links and images using Markdown syntax if present in the original document
- Preserve any special formatting or structure that's important to the document's meaning
Before providing the final Markdown output, in <document analysis> tags:
- Identify the main sections and subsections of the document
- Count the number of sections and subsections to ensure proper nesting of headers
This will help ensure a thorough and well-organized conversion.
After your analysis, present the converted document in Markdown format. Use ''' markers to denote the beginning and end of the Markdown content.
Example output structure:
<document analysis>
[Your analysis of the document structure and conversion plan]
</document analysis>
'''
markdown
# Document Title
## Section 1
Content of section 1...
## Section 2
Content of section 2...
- List item 1
- List item 2
[Link text](https://example.com)
![[Image description]](image-url.jpg)
Please proceed with your analysis and conversion of the document.
```

The key insight is that while users can accomplish these tasks on their own, they'll get more consistent and higher-quality results when using prompts that have been carefully developed and tested by the MCP server authors.

How Prompts Work

Prompts define a set of user and assistant messages that clients can use directly. When a client requests a prompt, your server returns a list of messages that can be sent straight to Claude.



The basic structure looks like this:

- Define prompts using the `@mcp.prompt()` decorator
- Add a name and description for each prompt
- Return a list of messages that form the complete prompt

- These prompts should be high quality, well-tested, and relevant to your MCP server's purpose

Building a Format Command

Here's how to implement a document formatting prompt. First, you'll need to import the base message types:

```
from mcp.server.fastmcp import base
```

Then define your prompt function:

```
@mcp.prompt(
    name="format",
    description="Rewrites the contents of the document in
Markdown format."
)
def format_document(
    doc_id: str = Field(description="Id of the document to
format")
) -> list[base.Message]:
    prompt = f"""
Your goal is to reformat a document to be written with
markdown syntax.
```

The id of the document you need to reformat is:

```
{doc_id}
```

Add in headers, bullet points, tables, etc as necessary. Feel free to add in extra formatting.

Use the 'edit_document' tool to edit the document. After the document has been reformatted...

```
"""
```

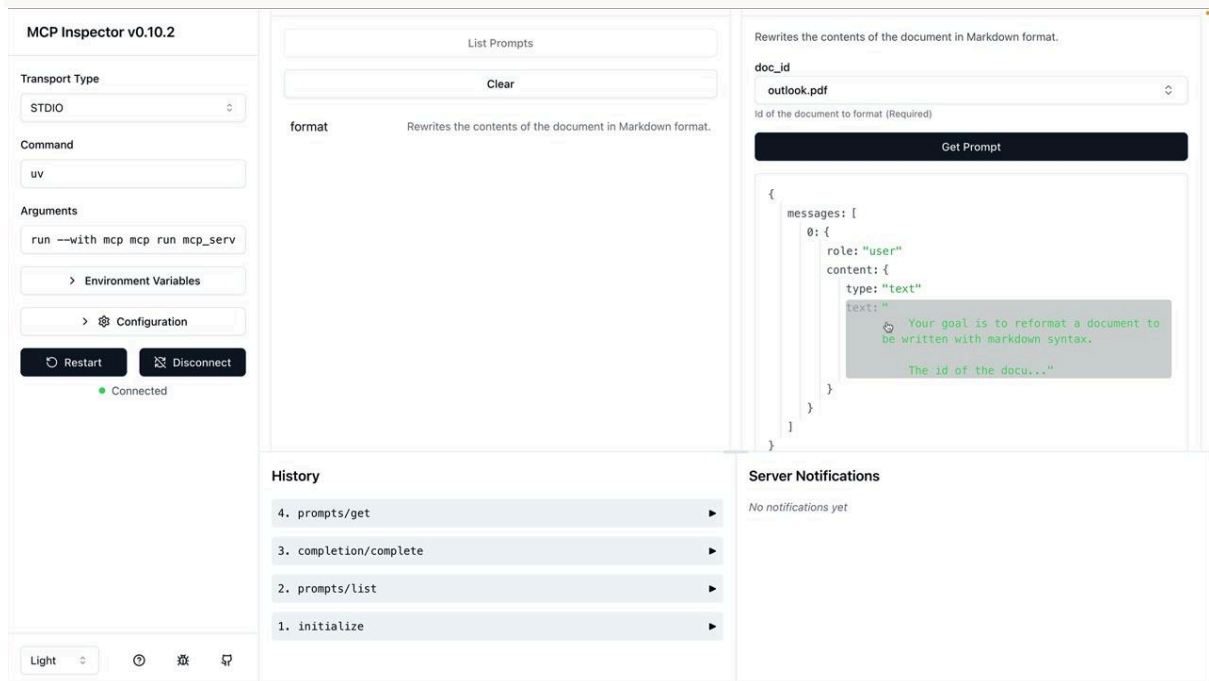
```
    return [
```

```
base.UserMessage(prompt)
```

```
]
```

Testing Your Prompts

You can test prompts using the MCP Inspector. Navigate to the Prompts section, select your prompt, and provide any required parameters. The inspector will show you the generated messages that would be sent to Claude.



This lets you verify that your prompt interpolates variables correctly and produces the expected message structure before using it in a real application.

Best Practices

When creating prompts for your MCP server:

- Focus on tasks that are central to your server's purpose
- Write detailed, specific instructions rather than vague requests
- Test your prompts thoroughly with different inputs
- Include clear descriptions so users understand what each prompt does
- Consider how the prompt will work with your server's tools and resources

Remember that prompts are meant to provide value that users couldn't easily get on their own - they should represent your expertise in the domain your MCP server covers.

Prompts in the client

Prompts in MCP define a set of user and assistant messages that can be used by the client. These prompts should be high quality, well-tested, and relevant to the overall purpose of the MCP server.

Prompts

- Defines a set of User and Assistant messages that can be used by the client
- These prompts should be high quality, well-tested, and relevant to the overall purpose of the MCP

ANTHROPIC

MCP Server

Prompt

```
@mcp.prompt(
    name="format",
    description="Rewrites the contents of
a document in Markdown format",
)
def format_document(
    doc_id: str,
) -> list[base.Message]:
    # Return a list of messages
```

Implementing List Prompts

The first step is implementing the **list_prompts** method in your MCP client. This method retrieves all available prompts from the server:

```
async def list_prompts(self) -> list[types.Prompt]:
    result = await self.session().list_prompts()

    return result.prompts
```

This simple implementation calls the session's **list_prompts** method and returns the prompts array from the result.

Getting Individual Prompts

The **get_prompt** method retrieves a specific prompt with arguments interpolated into it. When you request a prompt, you provide arguments that get passed to the prompt function as keyword arguments:

```
async def get_prompt(self, prompt_name, args: dict[str, str]):  
    result = await self.session().get_prompt(prompt_name,  
args)  
  
    return result.messages
```

The method returns the messages from the result, which form a conversation that can be fed directly into Claude.

How Prompt Arguments Work

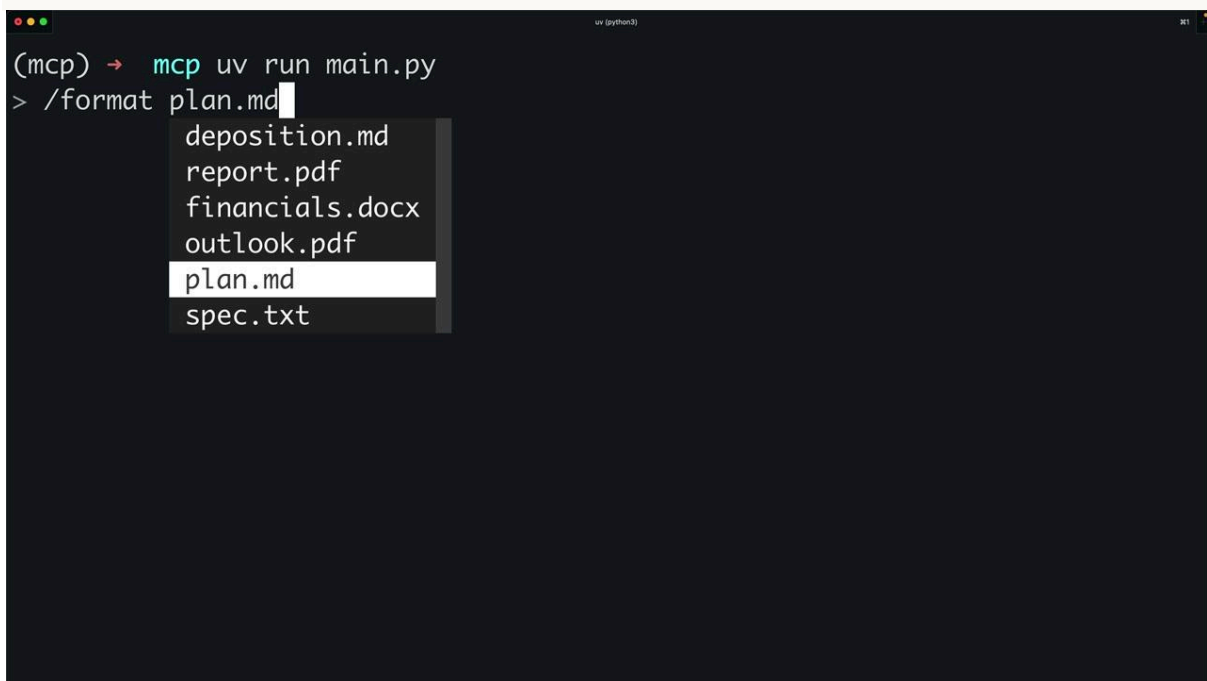
When you define a prompt function on the server side, it can accept parameters. For example, a document formatting prompt might expect a **doc_id** parameter:

```
def format_document(doc_id: str):  
  
    # The doc_id gets interpolated into the prompt
```

When the client calls `get_prompt`, the arguments dictionary should contain the expected keys. The MCP server will pass these as keyword arguments to the prompt function, allowing dynamic content to be inserted into the prompt template.

Testing Prompts in the CLI

Once implemented, you can test prompts through the command-line interface. When you type a forward slash, available prompts appear as commands. Selecting a prompt may prompt you to choose from available options (like document IDs), and then the complete prompt gets sent to Claude.



```
(mcp) -> mcp uv run main.py
> /format plan.md
  deposition.md
  report.pdf
  financials.docx
  outlook.pdf
  plan.md
  spec.txt
```

The workflow looks like this:

1. User selects a prompt (like "format")

2. System prompts for required arguments (like which document to format)
3. The prompt gets sent to Claude with the interpolated values
4. Claude can then use tools to fetch additional data and complete the task

```
uv (python3) M1

| Resource Type | Allocation | Timeline |
|-----|-----|-----|
| Development Team | 5 FTE | Phases 1-5 |
| QA Engineers | 2 FTE | Phases 3-4 |
| Project Management | 1 FTE | All Phases |
| Infrastructure | Cloud-based | Continuous |

## Risk Management

- **Identified Risks**:
  - Timeline delays
  - Scope creep
  - Technical challenges
  - Resource constraints

- **Mitigation Strategies**:
```

Prompt Best Practices

When creating prompts for your MCP server:

- Make them relevant to your server's purpose
- Test them thoroughly before deployment
- Use clear, specific instructions
- Design them to work well with your available tools
- Consider what arguments users will need to provide

Prompts bridge the gap between predefined functionality and dynamic user needs, giving Claude structured starting points for complex tasks while maintaining flexibility through parameterization.